

실습 1차시

AI에게 잘 시키는 법

학습 목표

- ✓ 맥락 + 제약조건 + 형식 구조로 프롬프트를 작성할 수 있어요
- ✓ 같은 요청에 대해 구조화 전/후의 응답 품질 차이를 비교할 수 있어요
- ✓ 자기 업무에 맞는 기본 프롬프트 3개를 구성할 수 있어요

같은 AI인데 누구는 원하는 답을 받고, 누구는 쓸모없는 답만 받아요. 차이는 AI의 성능이 아니라 **시키는 방법**에 있어요. 오늘은 같은 요청을 세 가지 방식으로 보내고, 결과가 얼마나 달라지는지 직접 확인해볼게요.

오늘 실습이 끝나면 여러분은 “맥락 + 제약조건 + 형식”이라는 세 축으로 프롬프트를 설계하는 법을 익히게 돼요. 이 구조만 잡아도 AI 응답의 품질이 눈에 띄게 달라져요.

준비물

- ChatGPT (무료 버전) 또는 Claude (무료 버전), 둘 중 하나만 있으면 돼요
- 웹 브라우저 (Chrome, Edge, Safari 등)
- 결과를 기록할 메모장 또는 노트 앱

프롬프트가 뭐길래

프롬프트(prompt)는 AI에게 보내는 지시문이에요. 식당에서 주문할 때를 생각해 보세요. “뭐 맛있는 거 주세요”라고 하면 셰프가 알아서 아무거나 내놓겠죠? “매운 거 빼고, 해산물 파스타, 크림 소스로 해주세요”라고 하면 원하는 음식이 나올 확률이 훨씬 높아요.

AI도 똑같아요. 프롬프트가 모호하면 AI는 그럴듯하지만 원하지 않는 답을 줘요. 프롬프트가 구체적 이면 원하는 답에 훨씬 가까워져요.

좋은 프롬프트에는 세 가지 축이 있어요.

축	설명	식당 비유
맥락(Context)	상황, 배경, 관계 등 작업의 배경 정보를 제공	알레르기, 취향, 예산, 몇 명인지 알려주기
제약조건(Constraints)	지켜야 할 규칙, 포함/제외할 내용을 명시	맵기 빼고, 해산물 위주로 해달라고 요청
형식(Format)	출력의 구조와 형태를 지정	코스 요리인지, 단품인지 정하기

이 세 축을 하나씩 추가하면서 결과가 어떻게 달라지는지 직접 확인해볼게요.

실습: 프롬프트 3단 변환

1 나쁜 프롬프트로 시도하기

먼저 가장 간단한 프롬프트로 시작해볼게요. ChatGPT 또는 Claude를 열고 아래 프롬프트를 그대로 입력해보세요.

사과 메일 써줘

AI가 뭔가를 써줄 거예요. 하지만 이 결과를 실제로 쓸 수 있을까요? 아마 여러 가지 문제가 보일 거예요.

결과를 받으면 아래 항목을 체크해보세요.

- 누구에게 보내는 메일인지 맞나요?
- 어떤 상황의 사과인지 맞나요?
- 메일의 톤이 적절한가요?
- 바로 복사해서 보낼 수 있는 수준인가요?

대부분의 경우 네 항목 중 하나도 만족하지 못할 거예요. AI는 아무 정보도 받지 못했으니 전부 추측으로 채울 수밖에 없거든요.

결과를 메모장에 복사해두세요. 나중에 비교할 거예요.

2 작업 맥락을 추가해보기

이번에는 새 대화를 열고, 작업 맥락을 추가해볼게요.

온라인 쇼핑몰의 CS팀장이 고객에게 보내는 사과 메일을 써줘.
배송이 늦어진 상황이야.

Step 1의 결과와 비교해보세요. 달라진 점이 보이나요?

작업 맥락을 추가하면 AI는 상황에 맞는 관점에서 글을 써요. “CS팀장이 고객에게 보내는 사과 메일”이라는 맥락이 들어가면 AI는 누가 누구에게, 왜 쓰는 글인지를 이해하고 적절한 어투와 구조를 잡아줘요.

하지만 여전히 부족한 부분이 있어요. **구체적으로 어떤 상황인지, 어떤 조건을 지켜야 하는지** AI는 모르거든요. 결과를 메모장에 복사해두세요.

3 구체적 맥락 + 제약조건 + 형식을 전부 추가하기

이번에는 다시 새 대화를 열고, 맥락 + 제약조건 + 형식을 모두 제공해볼게요.

<context>

발신: 온라인 쇼핑몰 CS팀장

수신: 노트북을 주문한 고객 (이미 1회 항의 전화를 한 상태)

사안: 배송 예정일보다 5일 지연, 원인은 물류 센터 시스템 오류

</context>

<task>

위 상황에서 사과 + 보상안 메일을 작성해주세요.

- 구체적인 보상안(15% 할인 쿠폰)을 포함해주세요

</task>

<format>

- 제목 포함

- 3단 구조: 사과 → 원인 설명 → 보상안

- 300자 내외

- 정중하되 과도하게 비굴하지 않은 톤

</format>

이 결과는 어떨까요? Step 1, Step 2와 비교해보세요.

구체적인 맥락이 들어가면 AI는 더 이상 추측하지 않아요. 상황을 정확히 알고 있으니

구체적인 내용을 채울 수 있거든요. 형식이 들어가면 출력 구조까지 예측 가능해져요.

XML 태그(<context>, <task>, <format>)로 영역을 분리하면 AI가 각 부분의 역할을 명확히 이해해요.

같은 AI, 같은 주제인데 프롬프트 구조만 바꿨을 뿐이에요. 그런데 결과는 완전히 달라요. 이것이 프롬프트 구조의 힘이에요.

4 세 가지 결과를 나란히 비교하기

메모장에 복사해둔 세 가지 결과를 나란히 놓고 비교해볼게요. 아래 표를 직접 채워보세요.

평가 항목	Step 1	Step 2	Step 3
상황 정확도	?/5	?/5	?/5
톤 적절성	?/5	?/5	?/5
바로 사용 가능 여부	?/5	?/5	?/5
구조/형식 완성도	?/5	?/5	?/5
총점	?/20	?/20	?/20

점수를 매겨보면 패턴이 보여요. 작업 맥락만 추가해도 점수가 오르고, 구체적인 제약 조건과 형식까지 추가하면 점수가 크게 뛰어요. 이게 “맥락 + 제약조건 + 형식”이라는 구조의 효과예요.

여기서 중요한 점은 이거예요. AI가 똑똑해져서 결과가 좋아진 게 아니예요. **사람이 정보를 잘 전달해서** 결과가 좋아진 거예요. AI의 성능을 끌어올리는 가장 빠른 방법은 모델을 바꾸는 게 아니라 프롬프트를 바꾸는 거예요.

심화: 내 업무에 적용하기

여기까지 따라왔다면 프롬프트 구조화의 원리를 이해한 거예요. 이제 본인의 실제 업무에 적용해볼 차례예요.

직접 해보기

아래 세 가지 상황 중 하나를 골라 “맥락 + 제약조건 + 형식” 구조로 프롬프트를 작성해보세요.

상황 A: 업무 메일 작성

- 맥락: 발신자, 수신자, 관계, 상황을 구체적으로 설명
- 제약조건: 반드시 포함할 내용, 제외할 내용
- 형식: 메일 제목 포함, 단락 수, 톤 지정

상황 B: 보고서 요약

- 맥락: 보고서의 주제, 핵심 수치, 독자가 누구인지
- 제약조건: 원본에 있는 수치만 사용, 추측 금지

- 형식: 3줄 요약, 핵심 수치 포함, 경어체

상황 C: 아이디어 브레인스토밍

- 맥락: 해결하려는 문제, 산업/분야, 현재 시도한 것
- 제약조건: 예산 범위, 실현 가능한 수준
- 형식: 번호 목록, 각 아이디어에 장단점 포함

작성한 프롬프트를 AI에 입력하고, 결과를 확인해보세요. 마음에 들지 않는 부분이 있다면 맥락이나 형식을 수정해서 다시 시도해보세요.

좋은 프롬프트의 체크리스트

프롬프트를 작성한 후 이 체크리스트로 점검해보세요.

항목	확인
작업 맥락이 충분한가요?	누가 누구에게, 왜, 어떤 상황에서 필요한 건지 설명했는지 확인하세요
구체적인 제약조건이 있나요?	포함할 내용, 제외할 내용, 지켜야 할 규칙을 명시했는지 확인하세요
이유(동기)를 함께 제공했나요?	“하지 마” 대신 “이렇게 해, 왜냐하면 “ 형태로 지시했는지 확인하세요
형식을 지정했나요?	길이, 구조, 톤 등 출력 형태를 정했는지 확인하세요
한 번에 하나만 요청했나요?	여러 작업을 한꺼번에 요청하면 품질이 떨어져요

자주 하는 실수

프롬프트를 처음 구조화할 때 자주 빠지는 함정이 있어요.

실수 1: 맥락 없이 막연하게 요청하기

“SNS 글 써줘”만으로는 부족해요. 어떤 제품인지, 타겟이 누구인지, 어떤 플랫폼에 올릴 건지, 톤은 어떤지를 알려줘야 해요. 작업의 구체적인 맥락이 없으면 AI는 모든 것을 추측으로 채울 수밖에 없어요.

실수 2: 한 번에 너무 많이 요청하기

“마케팅 전략 세우고, 예산도 잡고, 일정도 짜고, 보고서도 써줘.” 이렇게 하면 모든 항목이 알아져요. 한 번에 하나의 작업만 요청하고, 결과를 확인한 후 다음 작업을 요청하는 게 훨씬 효과적이에요.

실수 3: 형식 지정을 잊기

AI는 기본적으로 길게 쓰려는 경향이 있어요. “3줄로”, “표로”, “번호 목록으로” 같은 형식을 지정하지 않으면 불필요하게 긴 답변이 돌아와요. 형식을 지정하면 결과물의 활용도가 크게 올라가요.

실수 4: AI를 만능으로 기대하기

프롬프트를 아무리 잘 써도 AI는 최신 데이터를 모르거나, 사실을 지어내거나, 본인만의 내부 사정을 알 수 없어요. 프롬프트가 해결하는 것은 **전달의 문제**이지 **AI 능력의 한계**가 아니에요. 이 차이는 다음 차시에서 더 자세히 다룰 거예요.

트러블슈팅

Q. 맥락을 줬는데도 응답이 별로 달라지지 않아요.

A. 맥락이 아직 추상적일 수 있어요. '마케팅 글 써줘' 대신 '스타트업 B2B SaaS 제품의 인스타그램 캡션을 써줘. 타겟은 20대 개발자, 톤은 캐주얼'처럼 상황, 대상, 조건을 구체적으로 추가해보세요.

Q. 프롬프트를 길게 쓰면 AI가 지시를 무시해요.

A. 프롬프트가 너무 길면 AI가 일부를 놓칠 수 있어요. 핵심 지시는 프롬프트의 맨 앞이나 맨 뒤에 배치하세요. 중간에 있는 지시는 무시될 확률이 높아요.

Q. 같은 프롬프트를 넣었는데 매번 다른 답이 나와요.

A. 이건 정상이에요. AI는 확률적으로 텍스트를 생성하기 때문에 매번 조금씩 다른 결과가 나와요. 핵심 구조와 내용이 일관되면 정상적으로 작동하는 거예요.

Q. ChatGPT와 Claude에서 같은 프롬프트를 넣었는데 결과가 다른 이유는 뭔가요?

A. 모델마다 학습 데이터, 정렬 방식, 응답 스타일이 다르기 때문이에요. 같은 프롬프트라도 모델별로 다른 결과가 나오는 건 자연스러운 현상이에요. 프롬프트 구조가 좋으면 어떤 모델에서든 결과 품질이 높아져요.

Q. 형식을 지정했는데 AI가 안 지켜요.

A. 형식 지시는 프롬프트의 마지막에 배치하면 더 잘 지켜져요. '반드시 ~형식으로 작성해'처럼 강조하거나, 예시를 한 개 보여주면 효과적이에요.

핵심 정리

1. **맥락(Context)**을 제공하면 AI가 추측 대신 정보에 기반해서 답해요. 누가 누구에게, 왜, 어떤 상황에서 필요한지가 맥락의 핵심이에요.
2. **제약조건(Constraints)**을 명시하면 AI가 범위 안에서 정확하게 작업해요. “하지 마”보다 “이렇게 해”와 함께 이유를 제공하면 더 효과적이에요.
3. **형식(Format)**을 지정하면 출력 구조가 예측 가능해져요. 길이, 구조, 톤을 미리 정하면 바로 사용할 수 있는 결과물이 나와요.
4. 세 축을 동시에 적용하면 품질이 비약적으로 향상돼요. AI가 똑똑해진 게 아니라 **사람이 잘 전달한 것**이에요.

>> 다음 차시 예고**실습 2차시: 프롬프트의 한계 직접 체험**

프롬프트를 아무리 잘 써도 AI가 말을 안 듣는 순간이 와요. 대화가 길어지면 지시를 잊고, 컨텍스트가 없으면 엉뚱한 답을 해요. 이 한계를 직접 체험하고, 왜 프롬프트만으로는 부족한지 확인해 볼게요.

실습 2차시

프롬프트의 한계 직접 체험

학습 목표

- ✓ AI가 환각을 일으키는 상황을 의도적으로 재현할 수 있어요
- ✓ 프롬프트만으로 해결할 수 없는 한계 3가지를 식별할 수 있어요
- ✓ 한계를 극복하기 위해 컨텍스트 엔지니어링이 필요한 이유를 설명할 수 있어요

지난 시간에 역할 + 맥락 + 형식으로 프롬프트를 구조화하는 법을 배웠어요. 오늘은 한 걸음 더 나가 볼게요. 프롬프트를 아무리 잘 써도 AI가 말을 안 듣는 순간이 있다는 걸 직접 확인해볼 거예요.

이걸 아는 게 왜 중요할까요? **한계를 모르면 한계 안에서만 헤맨다**는 거예요. 프롬프트의 한계를 알아야 그 다음 단계, 즉 컨텍스트 엔지니어링으로 넘어갈 수 있어요.

준비물

- ChatGPT (무료 버전) 또는 Claude (무료 버전)
- 웹 브라우저
- 결과를 기록할 메모장 또는 노트 앱
- 시간 여유: 대화를 10턴 이상 이어갈 거예요

프롬프트의 세 가지 벽

프롬프트에는 구조적인 한계가 세 가지 있어요.

한계	원인	증상
지시 망각	대화가 길어지면 초기 지시가 밀려남	규칙을 점점 무시함
환각	모르는 정보를 확률적으로 생성	자신 있게 틀린 답을 줌
컨텍스트 부재	AI가 배경 정보 없이 추측	엉뚱한 방향으로 답변

오늘 실습에서는 이 세 가지를 모두 직접 체험해볼 거예요.

실습 A: 지시 망각 체험

1 시스템 프롬프트에 규칙 설정하기

새 대화를 열고, 첫 메시지에 아래 규칙을 입력하세요. 이것이 이 대화의 “규칙”이 돼요.

이 대화에서 너는 아래 규칙을 반드시 지켜야 해.

규칙:

1. "혁신적"이라는 단어를 절대 사용하지 마
2. 모든 답변에 구체적인 수치(% , 금액, 기간 등)를 반드시 포함해
3. 답변은 항상 3문장 이내로 작성해
4. 모든 답변의 끝에 "[규칙 준수]"라고 표시해

이해했으면 "규칙을 확인했습니다"라고 답해.

AI가 “규칙을 확인했습니다”라고 답하면 준비 완료예요.

2 5턴까지 규칙 준수 확인하기

이제 자연스러운 대화를 이어가세요. 어떤 주제든 상관없어요. 아래 예시를 참고하되, 본인만의 질문을 만들어봐도 좋아요.

- 턴 1: "한국 스타트업 시장의 현황을 알려줘"
 턴 2: "그중에서 가장 성장이 빠른 분야는?"
 턴 3: "AI 스타트업이 투자받으려면 뭘 준비해야 해?"
 턴 4: "실패하는 스타트업의 공통점은?"
 턴 5: "성공 사례를 하나 들어줘"

각 턴의 응답에서 네 가지 규칙이 지켜지고 있는지 체크하세요.

턴	혁신적 금지	수치 포함	3문장 이내	규칙 준수 표시
1				
2				
3				
4				
5				

보통 5턴까지는 규칙을 잘 지켜요. AI가 “규칙을 기억하고 있는 상태”이기 때문이에요.

3 10턴 넘긴 후 같은 규칙 확인하기

대화를 계속 이어가세요. 주제를 조금씩 바꾸면서 10턴을 넘겨보세요.

- 턴 6: "한국의 교육 시스템 문제점은?"
 턴 7: "AI가 교육에 미치는 영향은?"
 턴 8: "온라인 교육 플랫폼 추천해줘"
 턴 9: "자기계발 책 3권 추천해줘"
 턴 10: "시간 관리 방법 알려줘"
 턴 11: "한국의 부동산 시장 전망은?"
 턴 12: "투자 초보가 알아야 할 것은?"

10턴이 넘어가면 변화가 보이기 시작할 거예요. AI가 규칙 중 일부를 슬슬 잊기 시작해요.

흔히 나타나는 패턴은 이래요.

- “[규칙 준수]” 표시가 먼저 사라져요 (가장 약한 규칙)
- 3문장 제한이 풀려요 (자연스러운 응답 본능이 이김)
- “혁신적”이라는 단어가 슬쩍 등장해요
- 수치 포함 규칙이 마지막까지 남는 경우가 많아요

이것이 **지시 망각(instruction drift)** 현상이에요. 대화가 길어지면 AI의 “주의 (attention)”가 초기 지시에서 점점 멀어져요. 프롬프트 엔지니어링만으로는 이 문제를 근본적으로 해결할 수 없어요.

각 턴에서 규칙 위반이 언제 시작되는지 기록해두세요. 보통 7~10턴 사이에서 첫 위반이 나타나요.

실습 B: 컨텍스트 유무 비교

4 컨텍스트 없이 질문하기

새 대화를 열고 아래 프롬프트를 입력해보세요.

우리 회사 상황에 맞는 마케팅 전략을 3가지 제안해줘.

AI의 응답을 확인해보세요. 아마 이런 식의 답이 나올 거예요.

- “소셜 미디어 마케팅을 강화하세요”
- “콘텐츠 마케팅에 투자하세요”
- “인플루언서 마케팅을 활용하세요”

맞는 말이긴 한데, **모든 회사에 해당되는 일반론**이에요. “우리 회사”가 어떤 회사인지 AI는 전혀 모르니까요. 이런 답변은 검색 엔진 결과와 다를 게 없어요.

5 컨텍스트를 제공하고 같은 질문하기

새 대화를 열고, 이번에는 상세한 컨텍스트를 먼저 제공한 뒤 같은 질문을 해보세요.

우리 회사 정보:

- 업종: B2B SaaS (프로젝트 관리 도구)
- 직원 수: 15명
- 현재 고객: 국내 중소기업 50곳
- 월 매출: 3,000만 원
- 마케팅 예산: 월 300만 원
- 주요 경쟁사: Notion, Monday.com
- 강점: 한국어 UI, 국내 법인 지원
- 약점: 영어 지원 미비, 글로벌 인지도 없음
- 목표: 6개월 내 고객 100곳 달성

이 상황에 맞는 마케팅 전략을 3가지 제안해줘.
각 전략에 예상 비용과 기대 효과를 포함해줘.

결과가 얼마나 다른지 비교해보세요.

비교 항목	컨텍스트 없이	컨텍스트 포함
구체성	일반론	우리 회사 맞춤
실행 가능성	낮음	높음
수치 포함 여부	거의 없음	예산/기간 반영
바로 적용 가능 여부	불가	검토 후 적용 가능

같은 질문이지만 컨텍스트 유무에 따라 결과가 완전히 달라져요. 프롬프트 기법(역할, 형식)만으로는 **정보의 부재**를 메울 수 없어요. AI에게 판단 재료를 주는 것, 이것이 컨텍스트 엔지니어링의 시작이에요.

실습 C: 환각 유도 체험

6 존재하지 않는 정보 질문하기

새 대화를 열고 아래 프롬프트를 입력해보세요. 이 질문들은 **정답이 존재하지 않는** 질문이에요.

김소월 시인의 세 번째 시집 제목이 뭐야?

김소월은 생전에 시집을 한 권(진달래꽃, 1925)만 출간했어요. 세 번째 시집은 존재하지 않아요. 하지만 AI는 높은 확률로 그럴듯한 제목을 지어내서 답해요. 이것이 **환각 (hallucination)**이에요.

한 가지 더 시도해볼게요.

"Quantum Synapse Architecture"라는 논문의 주요 발견을 요약해줘.
이 논문은 2024년 Nature에 실렸어.

이 논문은 존재하지 않아요. 하지만 AI는 마치 읽어본 것처럼 상세한 요약을 만들어낼 수 있어요. 저자 이름, 실험 방법, 결론까지 지어내는 경우도 있어요.

왜 이런 일이 벌어질까요? AI는 “모른다”고 답하는 것보다 “그럴듯한 답을 생성하는 것”이 확률적으로 더 자연스럽다고 판단하기 때문이에요. AI는 검색 엔진이 아니에요. **확률적 텍스트 생성기**예요. 모르는 것에 대해 “모른다”고 말하도록 훈련받았지만, 완벽하지는 않아요.

환각은 AI의 버그가 아니라 **구조적 특성**이에요. 프롬프트를 아무리 잘 써도 환각 가능성을 0으로 만들 수는 없어요. 이것이 프롬프트 엔지니어링의 근본적 한계 중 하나예요.

한계의 지도 그리기

오늘 체험한 세 가지 한계를 정리해볼게요.

한계	프롬프트로 해결 가능?	해결 방향	다루는 차시
지시 망각	부분적	시스템 프롬프트, 컨텍스트 설계	실습 06
환각	완전 해결 불가	출처 요구, RAG, 검증 루프	실습 13
컨텍스트 부재	부분적	컨텍스트 엔지니어링	실습 05 06

프롬프트 엔지니어링은 강력하지만 만능이 아니에요. 프롬프트가 해결하는 것은 “어떻게 시킬 것인가”의 문제예요. “어떤 정보를 줄 것인가”, “어떻게 기억하게 할 것인가”는 프롬프트 너머의 문제예요. 이것이 컨텍스트 엔지니어링이 필요한 이유이고, 앞으로의 실습에서 하나씩 다뤄갈 거예요.

직접 해보기

본인의 업무에서 AI가 **잘 안 되는 상황**을 하나 떠올려보세요.

- 그 문제가 세 가지 한계 중 어디에 해당하나요?
- 프롬프트를 고치면 해결될까요, 아니면 다른 접근이 필요할까요?

메모장에 아래 형식으로 정리해보세요.

문제 상황: (예: AI에게 우리 회사 제품 설명을 시켰는데 엉뚱한 내용이 나옴)

한계 유형: (지시 망각 / 환각 / 컨텍스트 부재)

프롬프트로 해결 가능 여부: (O / X / 부분적)

필요한 추가 접근: (예: 제품 정보를 컨텍스트로 제공)

트러블슈팅

Q. 10턴이 넘어도 AI가 규칙을 잘 지키는데요?

A. 모델 성능이 올라가면서 지시 유지력도 좋아지고 있어요. 15턴, 20턴까지 이어가 보세요. 또는 주제를 크게 바꿔보세요. 규칙과 무관한 주제로 넘어가면 규칙 위반이 더 빨리 나타나요.

Q. 환각 유도 질문에 AI가 '모르겠다'고 답했어요.

A. 최근 모델들은 환각 방어가 많이 개선됐어요. 좋은 신호예요. 다만 항상 그런 것은 아니에요. 다른 없는 논문이나 없는 인물로 시도해보세요. 또는 '확실해? 나는 분명히 존재하는 걸로 아는데'라고 유도하면 환각이 나타나기도 해요.

Q. ChatGPT에서는 규칙을 잘 지키는데 Claude에서는 빨리 잊어요 (또는 반대).

A. 모델마다 지시 유지력이 다른 건 자연스러운 현상이에요. 중요한 것은 '모든 모델에 이 한계가 존재한다'는 사실을 아는 거예요. 정도의 차이일 뿐 모든 AI에 해당돼요.

Q. 컨텍스트를 많이 넣을수록 항상 좋은 건가요?

A. 관련 없는 정보를 너무 많이 넣으면 오히려 AI가 혼란스러워해요. *관련성 높은 정보를 구조화해서* 제공하는 게 중요해요. 양보다 질이에요.

Q. 환각을 완전히 없앨 방법은 없나요?

A. 프롬프트만으로는 불가능해요. 검증 가능한 출처를 요구하거나, 외부 데이터를 연결(RAG)하거나, 사람이 검토하는 루프를 추가하면 크게 줄일 수 있어요. 이걸 이후 차시에서 다뤄요.

핵심 정리

1. **지시 망각**: 대화가 7 10턴을 넘기면 AI는 초기 규칙을 잊기 시작해요. 프롬프트만으로는 완전히 해결할 수 없어요.
2. **환각**: AI는 모르는 것을 자신 있게 지어내는 구조적 특성이 있어요. 이것은 버그가 아니라 확률적 생성의 부산물이에요.
3. **컨텍스트 부재**: 배경 정보 없이 AI는 일반론으로 답할 수밖에 없어요. 프롬프트 기법이 아니라 정보 제공이 필요해요.
4. 이 세 가지 한계가 “프롬프트 엔지니어링 다음 단계”인 **컨텍스트 엔지니어링**이 필요한 이유예요.

>> 다음 차시 예고

실습 3차시: 업무 문서 AI로 만들기

프롬프트의 한계를 알았으니, 이제 한계 안에서 최대한 효과적으로 쓰는 법을 배워볼게요. 보고서 요약, 업무 메일, 회의록 정리까지 실무에서 바로 쓸 수 있는 문서 3종을 AI와 함께 만들어봐요.

실습 3차시

업무 문서 AI로 만들기

학습 목표

- ✓ 보고서 요약, 업무 메일, 회의록 정리 3종을 AI로 작성할 수 있어요
- ✓ 각 문서 유형에 맞는 프롬프트 템플릿을 구성할 수 있어요
- ✓ AI가 작성한 문서를 검토하고 수정하는 워크플로우를 실행할 수 있어요

지난 시간에 프롬프트의 한계를 확인했어요. 한계를 안다고 해서 AI를 안 쓸 수는 없잖아요. 오늘은 한계 안에서 최대한 효과적으로 활용하는 법을 다뤄볼게요.

오늘 만들 결과물은 세 가지예요. 보고서 요약, 업무 메일, 회의록 정리. 이 세 가지는 직장인이 가장 많은 시간을 쓰는 문서 작업이에요. AI에게 잘 시키면 30분짜리 작업을 5분으로 줄일 수 있어요.

준비물

- ChatGPT (무료 버전) 또는 Claude (무료 버전)
- 웹 브라우저
- 실습용 텍스트 (아래 제공), 본인의 실제 업무 텍스트가 있으면 더 좋아요
- 결과를 기록할 메모장 또는 노트 앱

AI 문서 작성의 원칙

AI에게 문서 작성을 시킬 때 기억해야 할 원칙이 세 가지 있어요.

원칙	설명
초안은 AI, 최종은 사람	AI가 만든 문서를 그대로 보내면 안 돼요. 반드시 검토와 수정을 거쳐야 해요
재료를 먼저 줘야 해요	AI는 없는 정보를 지어내요. 원본 데이터나 맥락을 먼저 제공해야 정확한 문서가 나와요
한 번에 하나씩	요약, 메일, 회의록을 한 프롬프트에 다 시키지 마세요. 하나씩 시켜야 품질이 올라가요

이제 문서 3종을 하나씩 만들어볼게요.

문서 1: 보고서 요약

1 긴 텍스트 붙여넣고 3줄 요약 요청하기

아래는 실습용 보고서 텍스트예요. 실제 업무 보고서가 있다면 그것을 써도 좋아요. (단, 사내 기밀이 포함된 문서는 무료 AI에 붙여넣지 마세요.)

[실습용 보고서 텍스트]

2026년 1분기 마케팅 성과 보고

1. 전체 요약

1분기 마케팅 총 지출은 4,500만 원으로 전년 동기 대비 15% 증가했다.
신규 고객 유치는 1,200명으로 목표(1,000명) 대비 120% 달성했다.
고객 획득 비용(CAC)은 37,500원으로 전분기 대비 8% 감소했다.

2. 채널별 성과

- 검색 광고: 지출 2,000만 원, 신규 고객 600명, CAC 33,333원
- SNS 광고: 지출 1,500만 원, 신규 고객 350명, CAC 42,857원
- 콘텐츠 마케팅: 지출 500만 원, 신규 고객 150명, CAC 33,333원
- 이메일 마케팅: 지출 200만 원, 신규 고객 80명, CAC 25,000원
- 오프라인 행사: 지출 300만 원, 신규 고객 20명, CAC 150,000원

3. 주요 이슈

- SNS 광고의 CAC가 전분기 대비 12% 상승, 크리에이티브 피로도 의심
- 이메일 마케팅이 CAC 최저, 확대 검토 필요
- 오프라인 행사 ROI가 현저히 낮음, 축소 또는 형태 변경 필요

4. 2분기 계획

- 검색 광고 유지, SNS 크리에이티브 리뉴얼
- 이메일 마케팅 예산 50% 증액
- 오프라인 행사 중단, 웨비나로 전환

이 텍스트를 AI에 붙여넣고 아래 프롬프트를 추가하세요.

위 보고서를 읽고 아래 형식으로 요약해줘.

형식:

- 3줄 요약 (각 줄에 핵심 수치 포함)
- 핵심 인사이트 1개 (데이터에서 도출)
- 의사결정 포인트 1개 (경영진이 바로 판단할 수 있게)

독자: 마케팅 예산을 승인하는 경영진

톤: 간결하고 데이터 중심

AI의 결과를 확인해보세요. 3줄 요약에 핵심 수치가 포함돼 있나요? 인사이트가 데이터에서 도출된 것인가요?

결과가 만족스럽지 않다면 아래를 시도해보세요.

- “SNS 광고 CAC 상승에 초점을 맞춰서 다시 요약해줘”
- “2분기 계획의 근거를 1분기 수치와 연결해줘”

이렇게 한 번의 요약 후 추가 지시를 주는 것을 **반복 다듬기(iterative refinement)**라고 해요. AI 문서 작성에서 가장 효과적인 방법이에요.

문서 2: 업무 메일

2 상황 컨텍스트 주고 메일 작성하기

새 대화를 열고 아래 프롬프트를 입력하세요.

<context>

발신: B2B SaaS 회사의 영업 담당자

수신: A사 김 부장님 (지난주 제품 데모 미팅 참석)

상황: 데모에서 "팀 협업 기능"과 "자동 보고서 생성"에 관심을 보임.

가격에 대해서는 "내부 검토가 필요하다"고 함.

이번 주 금요일까지 회신 받기로 했고, 지금은 수요일.

</context>

<task>

김 부장님에게 보낼 후속 메일을 작성해주세요.

- 데모에서 관심을 보인 기능을 자연스럽게 언급해주세요
- 금요일 회신을 부드럽게 리마인드해주세요
- 추가 자료(사례 소개서)를 첨부한다고 언급해주세요

</task>

<format>

- 제목 포함
- 200자 내외
- 정중하되 영업 냄새가 나지 않는 톤
- "안녕하세요, 김 부장님"으로 시작

</format>

결과를 확인해보세요. 아래 항목을 체크하세요.

- 데모에서 관심을 보인 기능이 언급돼 있나요?
- 금요일 리마인드가 부드럽게 들어가 있나요?
- 영업 냄새가 나지 않나요?

- 바로 복사해서 보낼 수 있는 수준인가요?

수정이 필요하면 구체적으로 요청하세요. “마지막 문장을 좀 더 부드럽게 바꿔줘”, “가 격 이야기는 빼줘” 같은 식이에요.

문서 3: 회의록 정리

3 회의 내용에서 액션아이템 추출하기

새 대화를 열고 아래 실습용 회의 내용을 붙여넣으세요.

[실습용 회의 내용, 날것 그대로의 메모]

2026년 4월 7일 주간 회의

참석: 이팀장, 김대리, 박사원, 정인턴

이팀장: 지난주 출시한 신기능 피드백이 들어오고 있어. 김대리가 정리해줘.

김대리: 네. 긍정 피드백이 70% 정도인데 UI가 복잡하다는 불만이 반복돼요.

특히 설정 메뉴 접근이 3단계인데 고객들이 못 찾아요.

이팀장: 설정 메뉴 접근을 2단계로 줄이는 게 가능해?

박사원: 기술적으로 가능한데 다음 스프린트에 넣으면 2주 걸려요.

이팀장: 그럼 다음 스프린트에 넣자. 김대리가 요구사항 정리하고

박사원이 기술 검토. 이번 주 금요일까지.

정인턴: 고객 인터뷰 5건 잡았는데 일정 조율 중이에요.

이팀장: 다음 주 수요일까지 완료해줘. 결과는 김대리한테 공유하고.

김대리: 경쟁사 A가 비슷한 기능을 출시했어요. 분석이 필요할 것 같아요.

이팀장: 정인턴이 경쟁사 분석 자료 만들어줘. 다음 주 회의에서 발표.

이 텍스트를 붙여넣고 아래 프롬프트를 추가하세요.

위 회의 내용을 아래 형식으로 정리해줘.

형식:

1. 회의 요약 (3줄)
2. 결정 사항 (번호 목록)
3. 액션아이템 테이블 (담당자 | 내용 | 마감일)
4. 다음 회의에서 확인할 사항

주의: 회의에서 명시적으로 언급된 내용만 포함해.

추측이나 추가 제안은 하지 마.

결과에서 특히 액션아이템 테이블을 확인하세요. 담당자, 내용, 마감일이 원본 회의 내용과 일치하나요?

“추측하지 마”라는 제약조건이 회의록 정리에서 특히 중요해요. 이 제약이 없으면 AI는 “추가로 이런 것도 하면 좋겠습니다”를 덧붙이는 경향이 있어요. 회의록은 사실 기록이지 제안서가 아니거든요.

결과물 검토와 수정

4 세 가지 결과물 검토하기

세 가지 문서(보고서 요약, 메일, 회의록)를 모두 만들었으면 검토 시간이에요. 아래 체크리스트로 각 문서를 점검해보세요.

검토 항목	보고서 요약	업무 메일	회의록
사실 오류가 없는가			
톤이 적절한가			
형식이 맞는가			
바로 사용 가능한가			
AI가 지어낸 내용이 있는가			

문제가 발견되면 AI에게 구체적으로 수정을 요청하세요. “전체적으로 다시 써줘”보다 “두 번째 문단의 톤을 좀 더 부드럽게 바꿔줘”가 훨씬 효과적이에요.

AI 문서 작성의 실전 워크플로우: **자료 제공 → 초안 생성 → 검토 → 구체적 수정 요청 → 최종 확인**. 이 다섯 단계가 하나의 사이클이에요. AI에게 한 번에 완벽한 문서를 기대하지 마세요. 반복 다듬기가 핵심이에요.

직접 해보기

지금까지 만든 세 가지 프롬프트를 **템플릿**으로 저장해보세요. 메모장이나 노트 앱에 아래 형식으로 정리하면 앞으로 재사용할 수 있어요.

[보고서 요약 템플릿]

<context> 입력: (보고서 붙여넣기) / 독자: ~ </context>

<task> 아래 형식으로 요약해주세요 </task>

<format> 3줄 요약 + 인사이트 1개 + 의사결정 포인트 1개 / 톤: ~ </format>

[메일 작성 템플릿]

<context> 발신: ~ / 수신: ~ / 상황: ~ / 관계: ~ </context>

<task> 위 상황에서 메일을 작성해주세요 </task>

<format> 제목 포함, ~자 내외, 톤: ~ </format>

[회의록 정리 템플릿]

<context> (여기에 회의 내용 붙여넣기) </context>

<task> 회의록을 정리해주세요 </task>

<format> 요약 + 결정사항 + 액션아이템 + 다음 확인사항 /

회의에서 명시적으로 언급된 내용만 포함, 추측 금지 </format>

본인의 업무 상황에 맞게 역할, 톤, 형식을 수정해서 자기만의 템플릿을 만들어보세요.

트러블슈팅

Q. 보고서 요약에서 원본에 없는 수치가 등장해요.

A. 이것이 바로 환각이에요. '원본에 있는 수치만 사용해'라는 제약조건을 프롬프트에 추가하세요. 그래도 발생하면 반드시 원본과 대조 확인하세요.

Q. 메일 톤이 너무 딱딱하거나 너무 가벼워요.

A. 톤 조절은 구체적인 예시가 효과적이에요. '비즈니스 캐주얼 톤, ~씨에게 보내는 것처럼'이라고 하거나, 원하는 톤의 메일 예시 한 개를 참고자료로 붙여넣으세요.

Q. 회의록에서 AI가 추가 제안을 자주 덧붙여요.

A. '회의에서 명시적으로 언급된 내용만 포함해. 추가 제안이나 의견은 절대 넣지 마.'라고 강하게 지시하세요. 그래도 넣는다면 '방금 결과에서 추가 제안 부분만 삭제해줘'라고 후속 지시를 보내세요.

Q. 사내 기밀 문서를 AI에 넣어도 되나요?

A. 무료 버전의 ChatGPT나 Claude는 입력 데이터가 모델 개선에 사용될 수 있어요. 사내 기밀은 절대 넣지 마세요. 기밀 문서 작업이 필요하다면 회사에서 승인한 AI 도구(엔터프라이즈 플랜)를 사용하거나, 민감 정보를 가명 처리한 후 입력하세요.

Q. 긴 보고서를 붙여넣으면 AI가 잘라먹어요.

A. 무료 버전은 입력 길이에 제한이 있어요. 보고서가 너무 길면 핵심 부분만 발췌해서 붙여넣거나, '이 보고서의 숫자 데이터 부분만 요약해줘'처럼 범위를 좁히세요.

핵심 정리

1. AI 문서 작성의 기본 원칙: **자료 제공 → 초안 생성 → 검토 → 수정 요청 → 최종 확인**의 5단계 사이클이에요.
2. **보고서 요약**은 원본 데이터를 통째로 제공하고, 독자와 형식을 명시하면 효과적이에요.
3. **업무 메일**은 상황 컨텍스트(관계, 히스토리, 목적)가 결과 품질을 좌우해요.
4. **회의록 정리**는 “추측하지 마”라는 제약조건이 핵심이에요. 사실 기록과 AI의 추가 제안을 명확히 분리해야 해요.

>> 다음 차시 예고

실습 4차시: 마케팅/기획에 AI 쓰기

업무 문서를 넘어서 마케팅과 기획 영역에서 AI를 활용해볼게요. SNS 캡션, 경쟁사 비교표, 사용자 페르소나, PRD 초안까지 네 가지 산출물을 만들어봐요.

실습 4차시

마케팅/기획에 AI 쓰기

학습 목표

- ✓ 마케팅/기획 산출물 4종을 AI로 작성할 수 있어요
- ✓ 톤과 타겟에 따라 프롬프트를 다르게 구성할 수 있어요
- ✓ AI 산출물을 실무 기준으로 검토하고 개선할 수 있어요

지난 시간에 보고서 요약, 메일, 회의록 같은 업무 문서를 AI로 만들어봤어요. 오늘은 한 걸음 더 나가서 마케팅과 기획 영역으로 확장해볼게요.

마케팅과 기획은 “정답이 없는 영역”이에요. 그래서 AI가 특히 유용해요. 사람이 방향을 잡고, AI가 다양한 변형을 빠르게 만들어주는 구조가 잘 맞거든요. 오늘은 SNS 캡션, 경쟁사 비교표, 사용자 페르소나, PRD 초안까지 네 가지를 만들어볼 거예요.

준비물

- ChatGPT (무료 버전) 또는 Claude (무료 버전)
- 웹 브라우저
- 본인이 관심 있는 제품/서비스 하나 (실습에서 활용)
- 결과를 기록할 메모장 또는 노트 앱

산출물 1: SNS 캡션 5종 생성

1 특별 SNS 캡션 만들기

같은 제품을 홍보하더라도 톤에 따라 캡션이 완전히 달라져요. 이것 직접 체험해볼게요.

새 대화를 열고 아래 프롬프트를 입력하세요.

<context>

플랫폼: 인스타그램

제품: 프로젝트 관리 앱 "TaskFlow"

특징: 팀원 간 실시간 협업, 자동 일정 조율, 진행률 대시보드

타겟: 20~30대 스타트업 직원

가격: 월 9,900원 (팀당)

</context>

<task>

아래 5가지 톤으로 인스타그램 캡션을 각각 1개씩 작성해주세요.

톤 1: 유머러스 (밈 느낌)

톤 2: 전문적 (B2B)

톤 3: 감성적 (워라밸 강조)
톤 4: 긴급성 (한정 이벤트)
톤 5: 사용자 후기 스타일
</task>

<format>

각 캡션은 3줄 이내, 해시태그 3개 포함.

</format>

결과를 보면 같은 제품인데 톤에 따라 완전히 다른 글이 나올 거예요. 이것이 AI를 마케팅에 쓰는 가장 큰 장점이예요. 사람이 5가지 톤을 모두 쓰려면 상당한 시간이 걸리지만, AI는 한 번의 요청으로 다양한 변형을 즉시 만들어줘요.

AI 마케팅 활용의 핵심: **사람이 방향(톤, 타겟)을 잡고, AI가 변형(variations)을 빠르게 생성하는 것**이예요. 100% 완성본을 기대하지 말고, 좋은 초안 후보를 빠르게 여러 개 얻는 도구로 활용하세요.

5개 캡션 중 마음에 드는 것을 하나 골라 다듬어보세요. “톤 3의 캡션에서 가격 정보를 자연스럽게 추가해줘”처럼 구체적으로 요청하면 돼요.

산출물 2: 경쟁사 비교표

2 경쟁사 비교표 작성하기

경쟁사 분석은 기획의 기본이예요. AI에게 비교표를 만들어달라고 해볼게요.

새 대화를 열고 아래 프롬프트를 입력하세요.

<context>

분석 목적: SaaS 프로젝트 관리 시장에서 우리 제품의 포지셔닝 파악

우리 제품:

- 이름: TaskFlow
- 카테고리: 프로젝트 관리
- 가격: 월 9,900원/팀
- 주요 기능: 실시간 협업, 자동 일정 조율, 진행률 대시보드
- 강점: 한국어 최적화, 국내 세금계산서 발행
- 약점: 글로벌 인지도 없음, 영어 UI 미지원

경쟁사: Notion, Monday.com, Asana

</context>

<task>

4개 제품(우리 포함)의 비교표를 만들어주세요.

- 비교 항목: 가격, 주요 기능, 한국어 지원, 타겟 시장, 강점, 약점
- 비교표 아래에 "우리 제품의 차별화 포인트" 2~3줄 추가

</task>

<format>

- 표 형식으로 작성
- 확실하지 않은 정보는 "(확인 필요)"라고 표시해주세요
(경쟁사 가격이나 기능은 수시로 바뀌므로)

</format>

여기서 중요한 것은 마지막 주의사항이에요. 경쟁사의 가격이나 기능은 수시로 바뀌니까 AI가 알고 있는 정보가 정확하지 않을 수 있어요. "(확인 필요)" 표시를 요청하면 AI가 확신 없는 정보를 표시해줘서, 나중에 사람이 확인할 수 있어요.

결과를 받으면 반드시 경쟁사 웹사이트에서 가격과 기능을 직접 확인하세요. AI가 제공한 비교표는 초안이지 최종본이 아니에요.

산출물 3: 사용자 페르소나

3 타겟 사용자 페르소나 만들기

페르소나는 타겟 사용자를 구체적인 인물로 형상화한 것이에요. 마케팅 전략, 제품 기획, 콘텐츠 제작의 기준점이 되어요.

새 대화를 열고 아래 프롬프트를 입력하세요.

<context>

제품: 프로젝트 관리 앱 "TaskFlow"

타겟: 스타트업에서 일하는 20~30대 직장인

목적: 마케팅 전략과 제품 기획의 기준점이 될 페르소나 작성

</context>

<task>

아래 구조로 사용자 페르소나 2명을 만들어주세요.

각 페르소나 구조:

1. 이름, 나이, 직업
2. 하루 일과 요약 (3줄)
3. 업무에서 느끼는 불만 3가지 (Pain Points)
4. 프로젝트 관리 도구에 기대하는 것 3가지 (Needs)
5. TaskFlow를 처음 접했을 때 할 만한 질문 2가지
6. 이 사람을 설득할 수 있는 한 줄 카피

</task>

<format>

- 두 페르소나는 서로 다른 직군이어야 합니다 (예: 개발자 + 마케터)
- 현실적이고 구체적인 디테일을 넣어주세요
- 너무 이상적인 사용자가 아니라, 실제로 있을 법한 사람으로 만들어주세요

</format>

결과를 보면 꽤 구체적인 인물이 나올 거예요. 하지만 여기서도 검토가 필요해요.

페르소나 검토 포인트:

- 이 사람이 실제로 존재할 것 같나요?
- Pain Points가 현실적인가요? (본인이나 주변 사람의 경험과 비교해보세요)
- 설득 카피가 이 사람에게 실제로 통할 것 같나요?

마음에 들지 않는 부분이 있다면 “첫 번째 페르소나의 Pain Points를 좀 더 구체적으로 바꿔줘. 예를 들어 ‘슬랙 알림이 너무 많아서 중요한 메시지를 놓치는 상황’ 같은 디테일을 넣어줘”라고 수정 요청하세요.

산출물 4: PRD 초안

4 PRD 초안 작성하기

PRD(Product Requirements Document)는 제품의 요구사항을 정리한 기획 문서예요. 보통 PM(Product Manager)이 작성하는데, AI가 초안을 잡아주면 시간을 크게 절약할 수 있어요.

새 대화를 열고 아래 프롬프트를 입력하세요.

<context>

제품: 프로젝트 관리 앱 "TaskFlow"

신규 기능 요청: "AI 기반 자동 업무 분배"

현재 상황: 팀장들이 매주 월요일 수동으로 업무를 배분, 평균 2시간 소요

목표: 업무 분배 시간을 30분 이내로 줄이기

문서 목적: 이 PRD는 엔지니어·디자이너와 공유할 기획 문서입니다

</context>

<task>

아래 구조로 PRD 초안을 작성해주세요.

PRD 구조:

1. 개요 (3줄: 무엇을, 왜, 누구를 위해)
2. 문제 정의 (현재 상황의 문제점 3가지)
3. 제안 솔루션 (기능 설명, 작동 방식)
4. 사용자 시나리오 (Before vs After 비교)
5. 성공 지표 (측정 가능한 KPI 3가지)
6. 범위 외 (이번에 다루지 않는 것)
7. 일정 추정 (개발 소요 기간)

</task>

<format>

- 기술 구현 상세는 제외 (엔지니어가 별도로 작성하기 때문)
- 각 섹션은 간결하게, 총 A4 1장 분량
- 실현 가능한 수준으로 작성 (AI가 만능이라는 전제 금지)

</format>

PRD는 다른 산출물보다 검토가 더 중요해요. AI는 기술적 현실성을 정확히 판단하기 어렵거든요. 결과를 받으면 아래 질문으로 검토하세요.

- 문제 정의가 실제 사용자의 문제와 일치하나요?
- 제안 솔루션이 기술적으로 실현 가능한가요?
- 성공 지표가 측정 가능한가요?
- “범위 외”에 빠진 항목이 있나요?

PRD에서 AI의 가치는 **구조를 잡아주는 것**이에요. 내용의 정확성은 사람이 검증해야 해요. 특히 일정 추정과 기술적 제약은 엔지니어와 반드시 확인하세요.

내 제품/서비스에 적용하기

직접 해보기

본인이 관심 있는 제품이나 서비스를 하나 정해서, 오늘 만든 4가지 산출물 중 하나를 직접만들어보세요.

추천 순서:

1. 관심 제품의 기본 정보를 정리해요 (이름, 특징, 타겟, 가격 등)
2. 위 Step 1 4의 프롬프트 중 하나를 골라요
3. 프롬프트의 제품 정보를 본인의 제품으로 바꿔요
4. AI에 입력하고 결과를 확인해요
5. 마음에 안 드는 부분을 구체적으로 수정 요청해요

실제 업무에서 쓰는 제품이면 더 좋아요. 결과물이 바로 활용 가능할 수 있거든요.

트러블슈팅

Q. SNS 캡션이 너무 한국적이지 않아요. 번역투 같아요.

A. 프롬프트에 '한국 20대가 인스타그램에서 실제로 쓰는 말투로 작성해줘'를 추가하세요. 또는 원하는 톤의 실제 캡션 예시를 1~2개 첨부하면 AI가 그 스타일을 따라해요.

Q. 경쟁사 비교표에서 정보가 틀린 것 같아요.

A. AI가 아는 경쟁사 정보는 학습 데이터 기준이라 현재 가격이나 기능과 다를 수 있어요. 비교표는 '구조의 틀'로 사용하고, 구체적인 수치와 기능은 반드시 공식 웹사이트에서 확인하세요.

Q. 페르소나가 너무 뻔해요.

A. '조금 엇지 있는 페르소나를 만들어줘. 예를 들어, 프로젝트 관리 도구를 싫어하는 사람이 어떻게 설득될 수 있는지 보여줘'라고 요청해보세요. 제약조건을 추가하면 더 흥미로운 페르소나가 나와요.

Q. PRD가 너무 추상적이에요.

A. 배경 정보를 더 구체적으로 제공하세요. '현재 팀장 3명을 인터뷰한 결과...'처럼 실제 데이터를 추가하면 PRD도 구체적으로 돼요. 재료가 추상적이면 결과도 추상적이에요.

Q. AI가 만든 결과물을 그대로 써도 되나요?

A. 안 돼요. AI 산출물은 항상 초안이에요. 특히 경쟁사 정보, 가격 데이터, 기술적 내용은 반드시 사람이 검증해야 해요. AI가 지어낸 정보가 섞여 있을 수 있거든요.

핵심 정리

1. **SNS 캡션:** 같은 제품도 톤에 따라 완전히 다른 글이 돼요. AI의 강점은 다양한 변형을 빠르게 만들어주는 것이에요.
2. **경쟁사 비교표:** AI가 만든 비교표는 구조의 틀이에요. 구체적인 수치는 반드시 사람이 확인해야 해요.
3. **사용자 페르소나:** 현실적인 디테일을 요청하면 품질이 올라가요. “실제로 있을 법한 사람”이 좋은 페르소나예요.
4. **PRD 초안:** AI는 구조를 잡아주고, 사람은 내용을 검증해요. 기술적 실현 가능성과 일정은 엔지니어와 확인하세요.

>> 다음 차시 예고**실습 5차시: 데이터 분석과 시각화**

마케팅/기획 산출물을 넘어서, 데이터 기반으로 일하는 방법을 배워볼게요. 엑셀 데이터를 AI에 붙여넣고 분석을 요청하고, 인사이트를 추출하고, 보고서 초안까지 만들어봐요.

실습 5차시

데이터 분석과 시각화

학습 목표

- ✓ 엑셀/CSV 데이터를 AI에 제공하여 분석을 요청할 수 있어요
- ✓ 데이터에서 의미 있는 인사이트를 추출하는 프롬프트를 작성할 수 있어요
- ✓ 분석 결과를 바탕으로 보고서 초안을 구성할 수 있어요

지난 시간에 마케팅/기획 산출물을 만들어봤어요. 오늘은 데이터를 다뤄볼게요. “데이터 분석”이라고 하면 어려워 보이지만, AI에게 데이터를 주고 “이게 무슨 의미야?”라고 묻는 것만으로도 상당한 인사이트를 얻을 수 있어요.

오늘 실습이 끝나면 엑셀 데이터를 AI에 붙여넣고, 분석을 요청하고, 인사이트를 뽑고, 보고서 초안까지 만드는 전체 흐름을 경험하게 돼요.

준비물

- ChatGPT (무료 버전) 또는 Claude (무료 버전)
- 웹 브라우저
- 실습용 데이터 (아래 제공), 본인의 실제 업무 데이터가 있으면 더 좋아요
- 결과를 기록할 메모장 또는 노트 앱

AI에게 데이터를 넘기는 법

AI에게 데이터를 넘기는 가장 간단한 방법은 **텍스트로 붙여넣기**예요. 엑셀에서 데이터를 복사하면 탭으로 구분된 텍스트가 되거든요. 이걸 그대로 대화창에 붙여넣으면 돼요.

데이터를 넘길 때 알아둘 점 세 가지가 있어요.

원칙	설명
열 이름을 명확히	A열, B열이 아니라 “월”, “매출”, “고객수” 같은 이름이 있어야 AI가 의미를 이해해요
단위를 표시해줘야 해요	숫자만 있으면 AI가 만원인지 원인지 %인지 모를 수 있어요. “매출(만원)” 같이 표시하세요
데이터 양에 한계가 있어요	무료 버전은 한 번에 넘길 수 있는 텍스트 양에 제한이 있어요. 100행 이내가 적당해요

실습 시작: 매출 데이터 분석

1 데이터 붙여넣고 분석 요청하기

아래 실습용 데이터를 복사해서 AI에 붙여넣으세요. 본인의 실제 업무 데이터가 있다면 그것을 사용해도 좋아요. (민감한 데이터는 가명 처리 후 사용하세요.)

월, 매출(만원), 신규고객(명), 이탈고객(명), 마케팅비(만원), NPS점수

2025-07, 3200, 120, 15, 450, 72

2025-08, 3100, 110, 22, 480, 68

2025-09, 3400, 135, 18, 520, 71

2025-10, 3800, 160, 12, 600, 75

2025-11, 4200, 180, 10, 650, 78

2025-12, 4500, 195, 8, 700, 82

2026-01, 4100, 170, 25, 680, 74

2026-02, 3900, 155, 30, 620, 70

2026-03, 4300, 185, 14, 710, 79

이 데이터를 붙여넣고 아래 프롬프트를 추가하세요.

위 데이터를 분석해줘.

분석 요청:

1. 전체 추세 요약 (3줄)
2. 눈에 띄는 이상치(anomaly)가 있다면 지적
3. 매출과 가장 상관관계가 높아 보이는 변수는?
4. 데이터에서 읽을 수 있는 경고 신호가 있다면?

주의:

- 데이터에 있는 숫자만 사용해
- 추측이 필요한 부분은 "(추측)"으로 표시해
- 인과관계와 상관관계를 구분해서 말해

결과를 확인해보세요. AI가 데이터에서 패턴을 찾아냈나요?

아마 이런 분석이 나올 거예요:

- 매출은 전반적 상승 추세이지만 2026년 1 2월에 하락했어요
- 2026년 1 2월에 이탈 고객이 급증했어요 (25명, 30명)
- 매출과 신규 고객 수의 상관관계가 높아 보여요
- 2월의 NPS 점수 하락(70점)은 경고 신호예요

AI가 돌려준 분석에서 “상관관계”와 “인과관계”를 구분하는 게 중요해요. “마케팅비를 늘리면 매출이 는다”는 상관관계일 수 있지만 인과관계는 아닐 수 있어요. AI는 이 구분을 놓치는 경우가 있으니 사람이 판단해야 해요.

2 차트 설명 요청하기

무료 AI는 대부분 차트를 직접 그려주지 못해요. 하지만 **차트 설명**은 해줄 수 있어요. 데이터를 어떤 차트로 시각화하면 좋을지, 그리고 그 차트에서 무엇이 보일지를 물어볼게요.

같은 대화에서 아래 프롬프트를 이어서 입력하세요.

위 데이터를 시각화한다면 어떤 차트를 쓰는 게 좋을지 추천해줘.

요청:

1. 차트 3개를 추천해줘 (종류, 무엇을 보여주는지, 축 설정)
2. 각 차트에서 핵심적으로 보이는 패턴을 설명해줘
3. 경영진 보고용으로 가장 효과적인 차트 1개를 고르고 이유를 설명해줘

형식: 번호 목록

AI가 추천하는 차트 종류를 확인해보세요. 보통 이런 추천이 나와요:

- 매출 추이 -> 꺾은선 그래프 (시간 축)
- 채널별 비교 -> 막대 그래프
- 매출 vs 마케팅비 -> 산점도

AI가 “이 차트에서 이런 패턴이 보인다”고 설명하는 부분이 핵심이에요. 실제로 차트를 만들기 전에 AI의 설명으로 **무엇을 보여줄 것인지를** 먼저 결정할 수 있거든요.

직접 해보기

AI가 추천한 차트 중 하나를 실제로 만들어보세요.

- 엑셀/구글 시트에 데이터를 넣고 차트를 생성해요
- AI의 설명과 실제 차트를 비교해보세요
- AI의 패턴 분석이 맞았나요?

3 인사이트 추출하기

데이터를 분석했으니, 이제 **의사결정에 필요한 인사이트**를 뽑아볼게요. 같은 대화에서 이어서 입력하세요.

지금까지의 분석을 바탕으로, 아래 질문에 답해줘.

1. 2026년 2분기에 가장 주의해야 할 지표는 무엇이고, 왜?
2. 마케팅 예산 배분을 조정한다면 어떻게 바꿀 것을 제안해?
(단, 총 예산은 현재와 동일하다는 전제)
3. 이탈 고객 급증(1~2월)의 가능한 원인 3가지를 추측해봐.
(이건 데이터만으로 알 수 없으니 "(추측)"이라고 명시해)

형식:

- 각 답변에 근거가 되는 수치를 반드시 포함
- 확실한 것과 추측인 것을 명확히 구분

결과에서 “확실한 것”과 “추측인 것”이 구분돼 있는지 확인하세요.

AI가 내놓는 인사이트의 가치는 **새로운 관점**에 있어요. 사람이 데이터를 보면 익숙한 패턴만 보게 되는 경향이 있어요. AI는 다른 각도에서 패턴을 제시해줘요. 물론 그 해석이 맞는지는 사람이 판단해야 해요.

AI 데이터 분석의 올바른 활용: **AI가 패턴을 발견하고, 사람이 의미를 판단하는 것**이에요. AI의 분석을 맹신하지 말고, “이 해석이 맞나?”라는 질문을 항상 달고 다니세요.

4 보고서 초안 작성하기

마지막으로, 지금까지의 분석을 하나의 보고서로 묶어볼게요. 같은 대화에서 이어서 입력하세요.

지금까지 분석한 내용을 바탕으로

"2026년 2분기 마케팅 전략 제안" 보고서 초안을 작성해줘.

보고서 구조:

1. 1분기 성과 요약 (핵심 수치 3개)
2. 주요 발견사항 (긍정 2개 + 경고 2개)
3. 2분기 전략 제안 (3가지, 각각 근거 수치 포함)
4. 모니터링 지표 (2분기에 주시할 KPI 3개)

독자: 마케팅 팀장

톤: 데이터 중심, 간결
분량: A4 1장 (500자 내외)
보고서 초안이 나오면 아래 체크리스트로 검토하세요.

검토 항목	확인
수치가 원본 데이터와 일치하나요?	AI가 수치를 잘못 인용하는 경우가 있어요
인과관계를 과도하게 주장하고 있나요?	“상관관계”를 “원인”으로 바꿔 쓰진 않았는지 확인
전략 제안이 실행 가능한가요?	현실적 제약(예산, 인력)을 반영했는지 확인
AI가 지어낸 수치가 있나요?	원본에 없는 숫자가 들어있진 않은지 대조
독자(마케팅 팀장)의 관점에서 유용한가요?	의사결정에 바로 활용할 수 있는 수준인지 확인

수정이 필요하면 구체적으로 요청하세요. “발견사항의 근거 수치를 더 정확하게 바꿔줘”, “전략 제안 3번이 너무 추상적이야. 구체적인 실행 방법을 추가해줘” 같은 식이에요.

데이터 분석의 전체 워크플로우

오늘 실습한 전체 흐름을 정리해볼게요.

단계	AI가 하는 일	사람이 하는 일
1. 데이터 전달	데이터를 읽고 구조 파악	깨끗한 데이터 준비, 단위 표시
2. 기초 분석	추세, 이상치, 상관관계 파악	분석 방향 지정, 질문 설계
3. 시각화 가이드	적합한 차트 추천, 패턴 설명	실제 차트 생성, 결과 확인
4. 인사이트	패턴 해석, 가설 제시	해석 검증, 의사결정

단계	AI가 하는 일	사람이 하는 일
5. 보고서	구조 잡기, 초안 작성	수치 검증, 최종 수정

이 워크플로우에서 AI는 “속도”를 제공하고, 사람은 “판단”을 제공해요. 이 역할 분담을 기억하세요.

직접 해보기

본인의 업무 데이터가 있다면, 위 4단계 워크플로우를 처음부터 끝까지 한 번 돌려보세요.

데이터가 없다면, 아래 추가 실습 데이터를 활용해보세요.

직원, 1월, 2월, 3월, 4월, 5월, 6월

김영희, 85, 88, 82, 90, 92, 95

이철수, 78, 72, 75, 80, 82, 79

박지영, 92, 95, 93, 91, 88, 85

정민수, 65, 70, 68, 72, 75, 80

한소희, 88, 85, 90, 87, 93, 95

이 데이터에 대해 “직원별 성과 추이 분석”을 요청해보세요.

트러블슈팅

Q. 데이터를 붙여넣었는데 AI가 표로 인식하지 못해요.

A. 데이터가 탭이나 쉼표로 구분돼 있는지 확인하세요. 구분자가 불명확하면 직접 표 형식을 알려주세요. '위 데이터에서 쉼표가 열 구분자이고 첫 줄이 헤더야'라고 명시하면 돼요.

Q. AI가 제시한 수치가 원본과 다른 것 같아요.

A. AI는 계산 실수를 할 수 있어요. 특히 평균, 증감률, 비율 같은 계산값은 반드시 직접 검증하세요. 엑셀에서 같은 계산을 해보는 게 가장 확실해요.

Q. AI가 통계 용어를 쓰는데 이해가 안 돼요.

A. '쉬운 말로 다시 설명해줘. 통계 용어는 쓰지 말고, 일상 언어로 바꿔줘'라고 요청하세요. AI는 설명 수준을 조절할 수 있어요.

Q. 데이터가 100행이 넘어서 AI에 못 넣겠어요.

A. 핵심 데이터만 추출하세요. 예를 들어 월별 요약, 상위/하위 10개만 발췌하는 식이에요. 또는 '이 데이터를 분석할 때 어떤 열이 가장 중요한지 알려줘'라고 먼저 물어본 뒤 핵심 열만 남기세요.

Q. AI가 인과관계를 마치 확정인 것처럼 말해요.

A. 프롬프트에 '상관관계와 인과관계를 구분해서 말해. 인과관계가 확인되지 않은 경우 반드시 (상관관계)라고 표시해'를 추가하세요. 그래도 넘어가는 경우가 있으니 사람이 최종 확인해야 해요.

핵심 정리

1. AI 데이터 분석은 **데이터 전달 → 기초 분석 → 시각화 가이드 → 인사이트 → 보고서**의 5단계 워크플로우를 따라요.
2. 데이터를 넘길 때 **열 이름, 단위, 데이터 구조**를 명확히 알려줘야 AI가 정확하게 분석해요.
3. AI의 분석에서 **상관관계와 인과관계를 구분**하는 것은 사람의 몫이에요. AI의 해석을 맹신하지 마세요.
4. AI는 **속도와 다양한 관점**을 제공하고, 사람은 **판단과 검증**을 담당해요. 이 역할 분담이 핵심이에요.

>> 다음 차시 예고**실습 6차시: 컨텍스트 설계 실습**

같은 작업에 컨텍스트를 4단계로 변형하면서, 컨텍스트가 AI 결과물에 미치는 영향을 체계적으로 비교해볼게요. 프롬프트 기법을 넘어서 컨텍스트 엔지니어링의 첫걸음을 내딛어봐요.

실습 6차시

컨텍스트 설계 실습

학습 목표

- ✓ 시스템 프롬프트에 페르소나 + 규칙 + 예시를 구조화하여 작성할 수 있어요
- ✓ 대화 기록 관리 전략(요약, 선택적 포함)을 실행할 수 있어요
- ✓ 동일 질문에 대해 컨텍스트 유/무 응답의 품질 차이를 비교할 수 있어요

지난 다섯 차시 동안 프롬프트를 구조화하고, 한계를 확인하고, 업무 문서와 마케팅 산출물과 데이터 분석까지 해봤어요. 오늘은 이 모든 것을 관통하는 근본 원리를 다뤄볼게요. 바로 **컨텍스트 설계**예요. 실습 2차시에서 컨텍스트 유무에 따라 결과가 크게 달라지는 걸 확인했었죠? 오늘은 그걸 더 체계적으로 실험해볼 거예요. 같은 작업에 컨텍스트를 4단계로 바꿔가면서, 결과가 어떻게 달라지는지 정밀 비교할 거예요.

준비물

- ChatGPT (무료 버전) 또는 Claude (무료 버전)
- 웹 브라우저
- 결과를 비교할 메모장 또는 노트 앱, 4개의 결과를 나란히 볼 거예요
- 시간 여유: 새 대화를 4번 열어야 해요

컨텍스트 엔지니어링이란

프롬프트 엔지니어링은 “어떻게 시킬 것인가”의 문제예요. 컨텍스트 엔지니어링은 “어떤 정보를 줄 것인가”의 문제예요.

이 차이를 식당 비유로 다시 생각해볼게요. 프롬프트 엔지니어링은 **주문서 작성법**이에요. “매운 거 빼고, 크림 파스타, 중간 사이즈”라고 적는 거죠. 컨텍스트 엔지니어링은 **셰프에게 주는 식재료 세트**예요. 같은 주문서라도 냉장고에 뭐가 있느냐에 따라 요리가 달라지잖아요.

AI에게 제공하는 컨텍스트는 크게 세 층으로 나눌 수 있어요.

층	내용	비유
역할 컨텍스트	AI가 어떤 전문가인지	셰프의 전공
상황 컨텍스트	배경, 관계, 히스토리	손님의 알레르기, 취향, 지난 방문 기록
제약 컨텍스트	금지사항, 형식, 톤 규칙	예산, 시간, 재료 제한

오늘 실습에서는 이 세 층을 하나씩 추가하면서 결과 변화를 관찰할 거예요.

실습: 4단계 컨텍스트 실험

오늘의 실험 과제는 이거예요: “분기 사업 전략 보고서 작성”. 같은 작업을 4번 시키는데, 매번 컨텍스트를 한 층씩 추가할 거예요.

1 컨텍스트 없이 작업하기 (레벨 0)

새 대화를 열고 아래 프롬프트를 입력하세요. 일부러 아무 배경 정보도 주지 않을 거예요.

2분기 사업 전략 보고서를 작성해줘.

이게 전부예요. AI가 어떤 결과를 내놓는지 보세요.

결과를 받으면 아래 항목을 5점 척도로 평가하세요.

- 우리 회사에 맞는 내용인가? (1 5)
- 실행 가능한 전략인가? (1 5)
- 구체적인 수치가 포함돼 있는가? (1 5)
- 바로 사용할 수 있는 수준인가? (1 5)
- 독창적인 인사이트가 있는가? (1 5)

아마 대부분 2점 이하일 거예요. AI는 “어떤 회사”인지 전혀 모르니까 모든 회사에 해당되는 일반론을 늘어놓을 수밖에 없어요.

결과를 메모장에 복사해두세요.

2 역할 컨텍스트만 추가하기 (레벨 1)

새 대화를 열고 역할만 추가해보세요.

역할: 너는 B2B SaaS 스타트업의 C00야.
이 회사의 주력 제품은 프로젝트 관리 도구이고,
직원 20명, 월 매출 5,000만 원 규모야.

2분기 사업 전략 보고서를 작성해줘.

레벨 0과 비교해보세요. 어떤 부분이 좋아졌나요?

역할 컨텍스트가 들어가면 AI는 “B2B SaaS”와 “스타트업”에 맞는 어휘와 관점을 사용하기 시작해요. 일반론에서 한 단계 구체적인 수준으로 올라가요.

하지만 여전히 부족한 게 있어요. 현재 상황, 경쟁 환경, 지난 분기의 성과 같은 구체적인 정보가 없으니 AI가 채울 수 있는 범위에 한계가 있어요.

같은 5개 항목으로 평가하고 메모장에 복사해두세요.

3 상황 + 관계 + 제약조건 추가하기 (레벨 2)

새 대화를 열고, 이번에는 상세한 상황 정보까지 제공하세요.

역할: 너는 B2B SaaS 스타트업 "TaskFlow"의 C00야.

회사 현황:

- 직원: 20명 (개발 10, 마케팅 4, 영업 3, 경영지원 3)
- 월 매출: 5,000만 원 (전분기 대비 +15%)
- 고객: B2B 중소기업 80곳
- MRR: 5,000만 원, ARR: 6억 원
- 현금: 8억 원 (런웨이 약 12개월)

1분기 성과:

- 신규 고객 25곳 유치 (목표 20곳 대비 125%)
- 이탈률 3.5% (전분기 5.2%에서 개선)
- NPS 78점 (전분기 72점에서 상승)
- 주요 성과: 대시보드 기능 출시로 대기업 고객 2곳 확보

과제:

- 경쟁사(Notion, Monday.com)가 한국 시장 진출 강화 중
- 시리즈 A 펀딩을 3분기에 계획 중 (목표 30억 원)
- 개발 인력 채용이 어려워 로드맵 지연 발생

보고서 독자: CEO와 이사회

톤: 데이터 기반, 솔직하되 긍정적

분량: A4 2장

2분기 사업 전략 보고서를 작성해줘.

이 결과는 어떤가요? 레벨 0, 레벨 1과 비교하면 차이가 확연할 거예요.

상황 컨텍스트가 들어가면 AI는 “우리 회사의 구체적인 숫자”를 기반으로 전략을 세웁니다. 경쟁 상황, 재무 상태, 채용 어려움 같은 현실적 제약이 반영돼요.

같은 5개 항목으로 평가하고 메모장에 복사해두세요.

4 결과 비교표 작성하기

메모장에 복사해둔 세 가지 결과를 나란히 놓고 비교표를 작성해보세요.

평가 항목 (5점 만점)	레벨 0	레벨 1	레벨 2
회사 적합도	?	?	?
실행 가능성	?	?	?
수치 구체성	?	?	?
즉시 사용 가능 여부	?	?	?
독창적 인사이트	?	?	?
총점	?/25	?/25	?/25

패턴이 보이나요? 컨텍스트가 한 층 추가될 때마다 점수가 뚜렷하게 올라가요.

여기서 핵심 교훈은 이거예요.

AI의 결과 품질은 **모델 성능보다 제공된 컨텍스트의 질**에 더 크게 좌우돼요. 더 비싼 모델을 쓰는 것보다 더 좋은 컨텍스트를 주는 것이 효과적일 때가 많아요. 이것이 컨텍스트 엔지니어링의 핵심 원리예요.

심화: 컨텍스트 설계 패턴

패턴 1: 페르소나 + 규칙 + 예시

가장 효과적인 컨텍스트 설계 패턴은 세 요소를 조합하는 거예요.

[페르소나]

너는 ~이야. ~한 경험이 있고, ~한 스타일로 일해.

[규칙]

1. 항상 ~해
2. 절대 ~하지 마
3. ~할 때는 ~형식으로

[예시]

좋은 출력 예시:

"~~~~~"

나쁜 출력 예시:

"~~~~~"

예시를 포함하면 AI가 원하는 스타일을 훨씬 정확하게 따라해요. “이런 식으로 써줘”라고 말로 설명하는 것보다 실제 예시를 보여주는 게 10배 효과적이에요.

직접 해보기

본인의 업무에 맞는 “페르소나 + 규칙 + 예시” 컨텍스트를 만들어보세요.

예를 들어:

- **고객 응대 봇**: CS 전문가 페르소나 + 답변 규칙(길이, 톤) + 좋은/나쁜 답변 예시
- **보고서 작성 도우미**: 업계 전문가 페르소나 + 형식 규칙 + 기존 보고서 예시
- **콘텐츠 작성 도우미**: 브랜드 보이스 페르소나 + 글쓰기 규칙 + 기존 콘텐츠 예시

작성한 컨텍스트를 AI에 입력하고, 같은 질문을 컨텍스트 유/무로 비교해보세요.

패턴 2: 대화 기록 관리

실습 2차시에서 대화가 길어지면 AI가 지시를 잊는다는 걸 확인했어요. 이에 대한 대응 전략이 두 가지 있어요.

전략	방법	적합한 상황
새 대화 시작	중요 규칙을 첫 메시지에 다시 넣고 새 대화를 시작	규칙이 절대 바뀌면 안 되는 경우
중간 요약 삽입	대화 중간에 “지금까지의 대화를 요약해줘” 후 새 대화에 요약만 넣기	긴 대화를 이어가야 하는 경우

두 번째 전략을 직접 해볼게요.

지금까지 우리가 논의한 내용을 아래 형식으로 요약해줘.

1. 결정된 사항 (번호 목록)
2. 아직 결정되지 않은 사항
3. 다음에 논의할 주제

이 요약은 새 대화에서 컨텍스트로 사용할 거야.

이렇게 AI에게 요약을 만들게 한 뒤, 새 대화를 열어 그 요약을 첫 메시지에 붙여넣으면 돼요. 대화의 핵심만 넘기고 불필요한 잡담은 버리는 셈이에요. 이게 “대화 기록 관리”의 기본이에요.

컨텍스트 설계 체크리스트

지금까지 배운 내용을 체크리스트로 정리해볼게요. 앞으로 AI에게 중요한 작업을 시킬 때 이 리스트를 확인하세요.

항목	확인 질문
역할	AI가 어떤 전문가로 행동해야 하는지 명시했나요?
배경 정보	작업에 필요한 회사/프로젝트/상황 정보를 제공했나요?
관계	누구를 위한 결과물인지 (독자, 상사, 고객) 알려줬나요?
제약조건	하면 안 되는 것, 지켜야 할 규칙을 설정했나요?
형식	출력의 구조, 길이, 톤을 지정했나요?
예시	좋은 결과물의 예시를 한 개라도 보여줬나요?
검증 요청	확실하지 않은 정보에 대해 표시하도록 요청했나요?

이 7개 항목을 모두 채우면 대부분의 작업에서 만족스러운 결과를 얻을 수 있어요.

트러블슈팅

Q. 컨텍스트를 많이 줬는데도 결과가 별로예요.

A. 양이 아니라 관련성이 중요해요. 작업과 무관한 정보를 많이 넣으면 오히려 AI가 혼란스러워해요. '이 작업에 가장 중요한 정보 3가지'만 골라서 제공해보세요.

Q. 매번 긴 컨텍스트를 입력하기 귀찮아요.

A. 자주 쓰는 컨텍스트는 템플릿으로 저장해두세요. 메모장에 '회사 정보 템플릿', '보고서 컨텍스트 템플릿' 같이 만들어두면 복사해서 붙여넣기로 해결돼요. 이후 차시에서 이것을 자동화하는 방법도 배울 거예요.

Q. 예시를 넣으면 AI가 예시를 그대로 베끼는 것 같아요.

A. 예시와 함께 '이 예시의 구조와 톤을 참고하되, 내용은 새롭게 작성해줘'라고 명시하세요. 예시를 2~3개 넣으면 특정 예시를 베끼는 대신 공통 패턴을 학습하는 경향이 있어요.

Q. 컨텍스트 설계에 시간을 많이 쓰면 AI를 안 쓰는 것보다 오래 걸리지 않나요?

A. 처음에는 시간이 좀 걸려요. 하지만 템플릿을 한 번 만들어두면 다음부터는 복붙이에요. 보고서 매달 쓰는 사람이라면, 한 번의 컨텍스트 설계가 12개월의 시간을 절약해줘요.

Q. 회사 기밀 정보를 컨텍스트로 넣어도 되나요?

A. 무료 AI는 입력 데이터가 모델 훈련에 사용될 수 있어요. 기밀 정보는 넣지 마세요. 기밀이 포함된 작업은 엔터프라이즈 플랜을 쓰거나, 민감 정보를 가명으로 바꾼 뒤 입력하세요.

핵심 정리

1. **컨텍스트 엔지니어링**은 “어떤 정보를 줄 것인가”의 문제예요. 프롬프트 기법보다 결과 품질에 더 큰 영향을 미쳐요.
2. 컨텍스트는 **역할 + 상황 + 제약**의 세 층으로 구성돼요. 한 층이 추가될 때마다 결과 품질이 눈에 띄게 올라가요.
3. **페르소나 + 규칙 + 예시** 패턴은 가장 효과적인 컨텍스트 설계 방법이에요. 특히 예시의 효과가 큰 편이에요.
4. 대화가 길어지면 **중간 요약 후 새 대화 시작**으로 컨텍스트를 관리할 수 있어요.

실습 1 6차시 돌아보기

여기까지 오느라 수고했어요. 지난 6차시 동안 배운 것을 한눈에 정리해볼게요.

차시	배운 것	핵심 키워드
실습 01	역할 + 맥락 + 형식으로 프롬프트 구조화	프롬프트 3축
실습 02	지시 망각, 환각, 컨텍스트 부재 체험	프롬프트의 한계

차시	배운 것	핵심 키워드
실습 03	보고서 요약, 메일, 회의록 작성	업무 문서 자동화
실습 04	SNS, 비교표, 페르소나, PRD 작성	마케팅/기획 산출물
실습 05	데이터 분석, 인사이트 추출, 보고서	데이터 기반 의사결정
실습 06	컨텍스트 4단계 실험, 설계 패턴	컨텍스트 엔지니어링

1 6차시는 “프롬프트와 컨텍스트의 기초”였어요. 7차시부터는 한 단계 더 나아가요. CLAUDE.md로 프로젝트 설정을 자동화하고, MCP 서버로 AI의 능력을 확장하고, Skills로 반복 작업을 자동화하는 단계로 넘어갈 거예요.

>> 다음 차시 예고

실습 7차시: CLAUDE.md + 프로젝트 설정

매번 같은 컨텍스트를 복붙하는 게 귀찮았죠? CLAUDE.md 파일에 프로젝트 규칙을 한 번 적어 두면 AI가 매번 자동으로 읽어요. 프로젝트별 AI 설정을 만들어봐요.

실습 7차시

CLAUDE.md와 프로젝트 설정

학습 목표

- ✓ CLAUDE.md 파일에 프로젝트 규칙과 스타일 가이드를 작성할 수 있어요
- ✓ Claude Code에서 프로젝트 설정을 구성하고 동작을 확인할 수 있어요
- ✓ CLAUDE.md 유/무에 따른 Claude 응답 일관성 차이를 비교할 수 있어요

? 핵심 질문

같은 AI에게 같은 질문을 해도 매번 다른 답이 나와요. 프로젝트마다 원하는 스타일과 규칙이 있는데, 어떻게 하면 AI가 이것 기억하고 따르게 할 수 있을까요?

0. 오늘 만들 결과물

이번 실습이 끝나면 여러분의 프로젝트 폴더에 `CLAUDE.md` 파일이 생겨요. 이 파일 하나가 Claude Code의 행동 방식을 완전히 바꿔요. 마치 신입 직원에게 회사 매뉴얼을 건네는 것과 같아요. 매뉴얼 없이 일하면 매번 물어봐야 하지만, 매뉴얼이 있으면 스스로 판단하고 움직이거든요.

배경지식: 컨텍스트 엔지니어링과 CLAUDE.md

이론편에서 컨텍스트 엔지니어링을 배웠어요. 프롬프트만으로는 한계가 있고, AI에게 더 풍부한 맥락을 제공해야 좋은 결과가 나온다는 내용이었죠. CLAUDE.md는 이 원리를 파일 하나로 구현한 거예요.

CLAUDE.md는 Claude Code가 프로젝트를 열 때 자동으로 읽는 설정 파일이에요. 여기에 적힌 내용이 모든 대화의 시스템 프롬프트처럼 작동해요. 빌드 명령어, 코딩 스타일, 프로젝트 구조 같은 정보를 한번 적어두면, 매번 설명하지 않아도 Claude가 알아서 따라요.

준비물

- Claude Code가 설치된 환경 (터미널에서 `claude` 명령어 실행 가능)
- 빈 프로젝트 폴더 1개 (아무 이름이나 괜찮아요)
- 텍스트 에디터 (VS Code 권장)

1. Claude Code 설치 확인

1 Claude Code가 잘 설치되어 있는지 확인해요

먼저 터미널을 열어보세요. macOS라면 Terminal, Windows라면 PowerShell이나 WSL 터미널을 사용해요.

아래 명령어를 입력해보세요.

```
claude --version
```

버전 번호가 출력되면 설치가 완료된 상태예요. 만약 “command not found”가 나온다면 아직 설치가 안 된 거예요.

직접 해보기

설치가 안 되어 있다면 아래 명령어로 설치해보세요.

```
npm install -g @anthropic-ai/claude-code
```

Node.js가 설치되어 있어야 해요. Node.js가 없다면 <https://nodejs.org>에서 먼저 설치하세요. LTS 버전을 권장해요.

설치 후 다시 `claude --version`을 실행해서 버전이 출력되는지 확인해보세요.

[스크린샷: 터미널에서 `claude --version`을 실행하고 버전 번호가 출력되는 화면]

설치가 확인되었으면 Claude Code를 한번 실행해볼게요. 터미널에서 아무 폴더나 들어가서 `claude`를 입력하면 대화형 인터페이스가 열려요.

```
cd ~/Desktop
claude
```

Claude Code가 실행되면 프롬프트가 바뀌면서 대화를 시작할 수 있어요. 간단한 질문을 해보세요.

> 안녕, 지금 어떤 폴더에서 실행되고 있는지 알려줘

Claude가 현재 디렉토리 경로를 알려줄 거예요. 이것이 Claude Code의 핵심이에요. Claude는 자기가 어떤 폴더에서 실행되었는지를 알고 있고, 그 폴더의 파일을 읽고 수정할 수 있어요.

[스크린샷: Claude Code 대화 인터페이스에서 현재 폴더를 물어보고 답변을 받는 화면]

Claude Code는 단순한 채팅이 아니에요. 실제 파일 시스템에 접근할 수 있는 AI 에이전트예요. 이 점이 웹 기반 Claude와 가장 큰 차이예요.

/exit을 입력해서 일단 나오세요. 다음 단계에서 본격적으로 프로젝트를 만들어볼 거예요.

2. 빈 프로젝트에서 시작

2 실습용 프로젝트 폴더를 만들어요

먼저 실습용 프로젝트 폴더를 만들어볼게요. 이 프로젝트는 간단한 할 일 관리 앱을 가질 거예요. 실제로 코드를 작성하지 않아도 괜찮아요. CLAUDE.md의 효과를 확인하는 것이 목표니까요.

```
mkdir ~/practice-todo-app
cd ~/practice-todo-app
```

프로젝트의 기본 구조를 잡아볼게요. 아래 명령어로 폴더와 파일을 만들어보세요.

```
mkdir src tests docs
touch src/main.py src/models.py src/utils.py
touch tests/test_main.py
touch README.md
```

이제 이 폴더에서 Claude Code를 실행해보세요.

```
claude
```

CLAUDE.md가 없는 상태에서 먼저 Claude에게 작업을 시켜볼 거예요. 이렇게 입력해보세요.

> src/main.py에 할 일 목록을 관리하는 기본 코드를 작성해줘

[스크린샷: CLAUDE.md 없이 Claude Code에게 코드 작성을 요청하는 화면]

Claude가 코드를 작성할 거예요. 결과를 잘 살펴보세요. 어떤 스타일로 코드를 작성했나요? 변수명은 영어인가요 한글인가요? 주석은 있나요? 타입 힌트는 사용했나요?

이 결과를 메모해두세요. 잠시 후 CLAUDE.md를 작성한 다음에 같은 요청을 다시 해볼 거예요. 차이가 확실하게 보일 거예요.

직접 해보기

Claude가 생성한 코드에서 다음 항목을 확인하고 메모해보세요.

- 코딩 스타일: PEP 8을 따르나요?
- 주석 언어: 영어인가요 한글인가요?
- 타입 힌트: 사용하나요?
- 에러 처리: try-except가 있나요?
- 테스트: 테스트 코드도 함께 작성했나요?

이 메모는 Step 4에서 비교할 때 사용할 거예요.

/exit으로 Claude Code를 종료하세요. 이제 CLAUDE.md를 작성할 차례예요.

3. CLAUDE.md 작성

3 CLAUDE.md를 직접 작성해요

CLAUDE.md는 프로젝트 루트에 위치하는 마크다운 파일이에요. Claude Code가 프로젝트를 열 때 이 파일을 자동으로 읽어요. 별도의 설정이나 등록 과정이 필요 없어요. 파일이 있으면 읽고, 없으면 안 읽어요. 그만큼 단순해요.

텍스트 에디터로 CLAUDE.md 파일을 만들어보세요.

```
code ~/practice-todo-app/CLAUDE.md
```

CLAUDE.md의 구조

CLAUDE.md에는 크게 세 가지를 적어요. 빌드/실행 명령어, 코드 스타일 규칙, 프로젝트 구조예요.

1) 빌드 명령어

프로젝트를 어떻게 실행하고, 테스트하고, 빌드하는지를 적어요. Claude가 코드를 수정한 후 테스트를 돌려보거나, 빌드가 깨지지 않았는지 확인할 때 이 정보를 사용해요.

아래 내용을 CLAUDE.md에 작성해보세요.

Practice Todo App

빌드 & 실행

- 실행: ``python src/main.py``
- 테스트: ``pytest tests/ -v``
- 린트: ``ruff check src/``
- 포맷: ``ruff format src/``

빌드 명령어를 적어두면 Claude가 코드를 수정한 후 자동으로 테스트를 실행해요. “테스트 돌려 봐”라고 별도로 말하지 않아도 되는 거예요.

2) 코드 스타일 규칙

여러분이 원하는 코딩 스타일을 구체적으로 적어요. 모호하게 적으면 Claude도 모호하게 따르고, 구체적으로 적으면 정확하게 따라요.

CLAUDE.md에 이어서 작성해보세요.

코드 스타일

- 언어: Python 3.11+
- 타입 힌트: 모든 함수에 필수
- Docstring: Google 스타일, 한글로 작성
- 변수명: snake_case, 의미가 명확한 영어
- 주석: 한글로 작성, "왜"를 설명 (무엇을 하는지는 코드로)
- 에러 처리: 구체적 예외 타입 사용 (bare except 금지)
- import 순서: 표준 라이브러리 → 서드파티 → 로컬 (빈 줄로 구분)

3) 프로젝트 구조

프로젝트의 폴더 구조와 각 파일의 역할을 적어요. Claude가 “어디에 코드를 넣어야 하지?”라고 고민하지 않게 해주는 거예요.

프로젝트 구조

```
practice-todo-app/
+-- src/
|   +-- main.py          # 엔트리포인트, CLI 인터페이스
|   +-- models.py        # 데이터 모델 (Todo, TodoList)
|   +-- utils.py          # 유틸리티 함수
+-- tests/
|   +-- test_main.py      # 테스트
+-- docs/                 # 문서
+-- CLAUDE.md             # Claude Code 설정
+-- README.md             # 프로젝트 소개
```

규칙

- 새 기능 추가 시 반드시 테스트도 함께 작성

- 한 함수는 30줄을 넘기지 않음
- 전역 변수 사용 금지

[스크린샷: VS Code에서 CLAUDE.md 파일을 작성하고 있는 화면]

전체 CLAUDE.md 완성본

지금까지 작성한 내용을 합치면 아래와 같아요. 전체를 한번에 복사해서 사용해도 괜찮아요.

Practice Todo App

빌드 & 실행

- 실행: ``python src/main.py``
- 테스트: ``pytest tests/ -v``
- 린트: ``ruff check src/``
- 포맷: ``ruff format src/``

코드 스타일

- 언어: Python 3.11+
- 타입 힌트: 모든 함수에 필수
- Docstring: Google 스타일, 한글로 작성
- 변수명: snake_case, 의미가 명확한 영어
- 주석: 한글로 작성, "왜"를 설명 (무엇을 하는지는 코드로)
- 에러 처리: 구체적인 예외 타입 사용 (bare except 금지)
- import 순서: 표준 라이브러리 → 서드파티 → 로컬 (빈 줄로 구분)

프로젝트 구조

```
practice-todo-app/
+-- src/
|   +-- main.py          # 엔트리포인트, CLI 인터페이스
|   +-- models.py        # 데이터 모델 (Todo, TodoList)
|   +-- utils.py          # 유틸리티 함수
+-- tests/
|   +-- test_main.py      # 테스트
+-- docs/                 # 문서
+-- CLAUDE.md             # Claude Code 설정
+-- README.md             # 프로젝트 소개
```

규칙

- 새 기능 추가 시 반드시 테스트도 함께 작성
- 한 함수는 30줄을 넘기지 않음
- 전역 변수 사용 금지

CLAUDE.md는 “완벽해야 한다”는 부담을 가질 필요가 없어요. 처음에는 간단하게 시작하고, 프로젝트가 진행되면서 점점 내용을 추가하면 돼요. Claude와 협업하면서 “이것도 적어둬야겠다”는 생각이 들 때마다 업데이트하면 되는 거예요.

CLAUDE.md 작성 팁

CLAUDE.md를 효과적으로 작성하기 위한 팁 몇 가지를 정리해볼게요.

원칙	설명
구체적으로	“좋은 코드를 작성해” 대신 “타입 힌트 필수, Google 스타일 docstring”처럼 구체적으로 적어요
짧게	CLAUDE.md가 너무 길면 오히려 효과가 떨어져요. 핵심만 간결하게 적어요
실행 가능하게	빌드 명령어는 복사해서 바로 실행할 수 있는 형태로 적어요
점진적으로	처음부터 완벽하게 적을 필요 없어요. 필요할 때마다 추가하면 돼요
예시 포함	코드 스타일은 좋은 예시와 나쁜 예시를 함께 적으면 효과적이에요

4. CLAUDE.md 유무로 결과 비교

4 CLAUDE.md의 효과를 직접 눈으로 확인해요

이제 CLAUDE.md가 있는 상태에서 같은 요청을 다시 해볼 거예요. Step 2에서 메모해 둔 결과와 비교해보세요.

프로젝트 폴더에서 Claude Code를 다시 실행하세요.

```
cd ~/practice-todo-app
claude
```

먼저 기존 src/main.py의 내용을 지워볼게요.

> src/main.py 내용을 전부 지우고, 할 일 목록을 관리하는 기본 코드를 새로 작성해줘

[스크린샷: CLAUDE.md가 있는 상태에서 같은 요청을 하고 결과를 받는 화면]

결과가 확연히 달라졌을 거예요. Step 2에서의 결과와 비교해보세요.

항목	CLAUDE.md 없을 때	CLAUDE.md 있을 때
타입 힌트	있을 수도, 없을 수도	모든 함수에 타입 힌트 포함
Docstring	영어 or 없음	Google 스타일, 한글
주석 언어	영어	한글
에러 처리	기본적 수준	구체적 예외 타입 사용
테스트	별도 요청 필요	함께 작성됨

직접 해보기

비교를 더 확실하게 해보고 싶다면 아래 실험도 해보세요.

1. CLAUDE.md의 주석 언어 규칙을 “영어로 작성”으로 바꾸고, 같은 요청을 해보세요
2. CLAUDE.md에 “모든 함수 앞에 사용 예시를 docstring에 포함”이라는 규칙을 추가하고, 새 함수를 만들어달라고 해보세요
3. CLAUDE.md에 “출력은 항상 JSON 형태로”라는 규칙을 추가하고, 데이터 모델을 만들어 달라고 해보세요

규칙 하나를 바꿀 때마다 Claude의 출력이 어떻게 달라지는지 관찰해보세요.

CLAUDE.md의 계층 구조

CLAUDE.md는 한 곳에만 있는 것이 아니예요. 세 가지 위치에 둘 수 있고, 각각 다른 범위로 작동해요.

위치	범위	용도
~/ .claude/CLAUDE.md	모든 프로젝트	개인 선호 (언어, 문체 등)
프로젝트 루트 CLAUDE.md	해당 프로젝트	빌드, 스타일, 구조 (팀 공유)
하위 폴더 CLAUDE.md	해당 폴더 하위	모듈별 특수 규칙

여러 위치에 CLAUDE.md가 있으면 모두 읽어요. 충돌하는 규칙이 있으면 더 가까운(하위) 파일이 우선해요. 이 구조를 잘 활용하면 팀 전체의 규칙은 루트에, 특정 모듈의 규칙은 하위 폴더에 분리할 수 있어요.

직접 해보기

홈 디렉토리에 개인용 CLAUDE.md를 만들어보세요.

```
mkdir -p ~/.claude
```

~/.claude/CLAUDE.md에 아래 내용을 적어보세요.

개인 설정

- 응답 언어: 한국어
- 코드 주석: 한글
- 설명은 간결하게, 코드는 상세하게

이후 어떤 프로젝트에서 Claude Code를 실행하든 이 설정이 기본으로 적용돼요.

자주 쓰는 CLAUDE.md 패턴

실무에서 자주 사용되는 CLAUDE.md 패턴을 몇 가지 소개할게요.

프론트엔드 프로젝트

코드 스타일

- 프레임워크: React 19 + TypeScript
- 스타일: Tailwind CSS (인라인 스타일 금지)
- 상태관리: Zustand
- 컴포넌트: 함수형만 사용, Props 인터페이스 필수
- 파일명: kebab-case (예: user-profile.tsx)

백엔드 API 프로젝트

코드 스타일

- 프레임워크: FastAPI
- DB: PostgreSQL + SQLAlchemy 2.0
- 모든 엔드포인트에 Pydantic 모델 사용
- 에러 응답: RFC 7807 Problem Details 형식

데이터 분석 프로젝트

코드 스타일

- 환경: Jupyter Notebook + pandas 2.x
- 셀 순서: 데이터 로드 → 전처리 → 분석 → 시각화
- 시각화: matplotlib, 한글 폰트 설정 필수
- 변수명: df_는 DataFrame, s_는 Series 접두사

CLAUDE.md는 프로젝트의 성격에 따라 완전히 달라져요. 프론트엔드, 백엔드, 데이터 분석, 각 도메인에서 중요한 규칙이 다르니까요. 위 예시를 참고해서 여러분의 프로젝트에 맞게 커스터마이징해보세요.

5. 정리 및 다음 단계

이번 실습 핵심 정리

- **CLAUDE.md**는 Claude Code의 행동을 결정하는 프로젝트 설정 파일이에요
- 빌드 명령어, 코드 스타일, 프로젝트 구조 세 가지를 담아요
- CLAUDE.md가 있으면 Claude의 출력이 일관되고, 프로젝트 규칙을 자동으로 따라요
- 세 가지 위치(홈, 프로젝트 루트, 하위 폴더)에서 계층적으로 작동해요

트러블슈팅

Q. **claude** 명령어가 실행되지 않아요

A. Node.js 18 이상이 설치되어 있는지 확인하세요. ``node --version``으로 확인할 수 있어요. npm 전역 설치 경로가 PATH에 포함되어 있는지도 확인하세요. ``npm config get prefix``로 경로를 확인할 수 있어요.

Q. **CLAUDE.md**를 만들었는데 Claude가 무시하는 것 같아요

A. 파일 이름이 정확히 ``CLAUDE.md``인지 확인하세요. 대소문자가 중요해요. ``claude.md``나 ``Claude.md``가 아닌 ``CLAUDE.md``여야 해요. 또한 파일이 프로젝트 루트에 있는지 확인하세요.

Q. Claude가 **CLAUDE.md**의 규칙을 일부만 따라요

A. CLAUDE.md가 너무 길거나 규칙이 모호하면 일부를 놓칠 수 있어요. 규칙을 줄여보고, 가장 중요한 것부터 위에 배치하세요. 구체적인 예시를 함께 적으면 준수율이 올라가요.

Q. 기존 프로젝트에 CLAUDE.md를 추가하고 싶은데 뭘 적어야 할지 모르겠어요

A. Claude Code에게 물어보세요. `이 프로젝트를 분석하고 CLAUDE.md를 만들어줘`라고 요청하면 프로젝트 구조를 파악해서 초안을 작성해줘요. 거기서 수정하면 훨씬 빨라요.

Q. 팀원들과 CLAUDE.md를 공유하고 싶어요

A. CLAUDE.md는 일반 파일이니까 Git에 커밋하면 돼요. 프로젝트 루트의 CLAUDE.md는 팀 공유용, `~/.claude/CLAUDE.md`는 개인 설정용으로 분리하세요.

실습 체크리스트

- ☐ Claude Code를 설치하고 `claude --version`으로 확인했다
- ☐ 빈 프로젝트에서 CLAUDE.md 없이 코드 생성을 요청하고 결과를 메모했다
- ☐ CLAUDE.md를 작성했다 (빌드 명령어 + 코드 스타일 + 프로젝트 구조)
- ☐ CLAUDE.md가 있는 상태에서 같은 요청을 하고 결과를 비교했다

- 여기까지 완료하면 이번 실습은 성공이에요! -

- ☐ [도전] 개인용 `~/.claude/CLAUDE.md`를 만들어봤다
- ☐ [도전] CLAUDE.md 규칙을 바꿔가며 출력 차이를 3회 이상 실험했다
- ☐ [도전] 자기 실무 프로젝트에 맞는 CLAUDE.md를 작성해봤다

>> 다음 차시 예고**실습 8차시: MCP: 외부 도구 연결**

Claude가 파일을 읽고, 웹을 검색하고, 데이터베이스에 접근하게 만들어볼 거예요.
MCP(Model Context Protocol)를 설치하고 직접 연결해보는 실습이에요.

실습 8차시

MCP: 외부 도구 연결

학습 목표

- ✓ MCP 서버를 로컬 환경에 설치하고 연결 상태를 확인할 수 있어요
- ✓ 파일시스템 MCP 도구를 Claude에서 실행할 수 있어요
- ✓ MCP 연결 오류 발생 시 로그를 읽고 원인을 진단할 수 있어요

? 핵심 질문

Claude는 똑똑하지만 혼자서는 파일을 검색하거나 외부 데이터를 가져올 수 없어요. 어떻게 하면 Claude에게 “눈과 손”을 달아줄 수 있을까요?

0. 오늘 만들 결과물

이번 실습이 끝나면 Claude가 파일시스템 MCP 서버를 통해 여러분 컴퓨터의 파일을 직접 검색하고 읽을 수 있게 돼요. 전에는 “이 파일 좀 읽어줘”라고 일일이 복사해 붙여야 했다면, 이제는 “프로젝트에서 TODO가 포함된 파일을 찾아줘”라고 말하면 Claude가 알아서 찾아요.

배경지식: MCP란 무엇인가

이론편에서 MCP(Model Context Protocol)를 배웠어요. MCP는 AI 모델이 외부 도구와 통신하기 위한 표준 프로토콜이에요. USB가 주변기기를 컴퓨터에 연결하는 표준이듯, MCP는 도구를 AI에 연결하는 표준이에요.

MCP에는 세 가지 핵심 요소가 있어요.

- **리소스(Resources)**: AI가 읽을 수 있는 데이터 (파일, DB 레코드 등)
- **도구(Tools)**: AI가 실행할 수 있는 기능 (파일 검색, API 호출 등)
- **프롬프트(Prompts)**: 미리 정의된 대화 템플릿

오늘은 이 중에서 **도구**를 중심으로 실습해볼 거예요.

준비물

- Claude Code (지난 실습에서 설치 완료)
- Node.js 18 이상
- 실습용 프로젝트 폴더 (지난 실습의 practice-todo-app 재사용 가능)

1. settings.json에 MCP 서버 추가

1 MCP 서버를 Claude Code에 등록해요

MCP 서버를 Claude Code에 연결하려면 설정 파일에 서버 정보를 추가해야 해요. Claude Code의 설정 파일은 ~/.claude/settings.json이에요.

먼저 이 파일이 있는지 확인해보세요.

```
cat ~/.claude/settings.json
```

파일이 없거나 비어 있어도 괜찮아요. 지금부터 만들 거예요.

오늘 사용할 MCP 서버는 **파일시스템 MCP 서버**예요. Anthropic이 공식으로 제공하는 기본 MCP 서버 중 하나예요. 이 서버를 통해 Claude가 지정된 폴더의 파일을 읽고, 검색하고, 목록을 조회할 수 있어요.

Claude Code에서 MCP 서버를 추가하는 가장 쉬운 방법은 `claude mcp add` 명령어를 사용하는 거예요.

```
claude mcp add filesystem -s user, npx -y @modelcontextprotocol/server-filesystem ~/practice-todo-app
```

이 명령어를 분해해볼게요.

부분	의미
<code>claude mcp add</code>	MCP 서버를 추가하는 명령어
<code>filesystem</code>	이 MCP 서버의 이름 (우리가 정하는 별칭)
<code>-s user</code>	사용자 전역 설정에 추가 (<code>-s project</code> 로 하면 프로젝트별 설정)
<code>--</code>	뒤에 오는 것이 실제 서버 실행 명령어라는 표시
<code>npx -y @model...</code>	파일시스템 MCP 서버 패키지를 다운로드해서 실행
<code>~/practice-todo-app</code>	Claude가 접근할 수 있는 폴더 경로

[스크린샷: 터미널에서 `claude mcp add` 명령어를 실행하는 화면]

명령어가 성공하면 ~/.claude/settings.json에 서버 정보가 추가돼요. 확인해볼게요.

```
cat ~/.claude/settings.json
```

아래와 비슷한 내용이 보여야 해요.

```
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-filesystem",
        "/home/사용자이름/practice-todo-app"
      ]
    }
  }
}
```

MCP 서버 설정은 JSON 파일에 저장돼요. 서버 이름, 실행 명령어, 인자, 이 세 가지가 핵심이에요. 수동으로 JSON을 편집해도 되지만, `claude mcp add` 명령어가 훨씬 편해요.

settings.json의 범위

settings.json도 CLAUDE.md처럼 두 가지 위치에 둘 수 있어요.

위치	범위	플래그
~/.claude/settings.json	모든 프로젝트	-s user
프로젝트/.claude/settings.json	해당 프로젝트	-s project

개인적으로 항상 사용하는 MCP 서버는 user 범위에, 특정 프로젝트에서만 쓰는 서버는 project 범위에 등록하면 돼요.

2. 파일시스템 MCP 연결 확인

2 MCP 서버가 제대로 연결되었는지 확인해요

이제 Claude Code를 실행해서 MCP 서버가 잘 연결되었는지 확인해볼게요.

```
cd ~/practice-todo-app
claude
```

Claude Code가 시작되면서 MCP 서버에 연결을 시도해요. 시작 메시지에서 MCP 서버 목록이 표시되는지 확인하세요.

[스크린샷: Claude Code 시작 화면에서 MCP 서버 연결 상태가 표시되는 화면]

MCP 서버가 잘 연결되면 Claude에게 도구 목록을 물어볼 수 있어요.

> 지금 사용할 수 있는 MCP 도구 목록을 보여줘

파일시스템 MCP 서버가 제공하는 도구들이 목록으로 나와요. 주요 도구는 아래와 같아요.

도구	기능
<code>read_file</code>	지정한 파일의 내용을 읽어요
<code>write_file</code>	파일에 내용을 써요
<code>list_directory</code>	폴더 안의 파일 목록을 조회해요
<code>search_files</code>	파일 이름이나 내용으로 검색해요
<code>get_file_info</code>	파일 크기, 수정 시간 등 메타데이터를 조회해요

간단한 테스트를 해볼게요. Claude에게 프로젝트 구조를 물어보세요.

> 이 프로젝트의 파일 구조를 보여줘

[스크린샷: Claude가 MCP를 통해 파일 목록을 조회하고 결과를 보여주는 화면]

Claude가 `list_directory` 도구를 사용해서 파일 목록을 가져온 뒤, 트리 구조로 정리해서 보여줄 거예요.

Claude Code 자체도 파일을 읽을 수 있지만, MCP는 “어떤 도구를 어떤 범위에서 사용할 수 있는지”를 명확하게 정의하는 프로토콜이에요. 이 표준화가 핵심이에요.

3. MCP로 파일 검색/읽기

3 MCP 도구를 활용해서 실제 작업을 해봐요

이제 MCP 도구를 사용해서 실제로 유용한 작업을 해볼 거예요. 먼저 프로젝트에 파일 몇 개를 더 만들어서 검색할 내용을 추가할게요.

Claude Code에서 아래와 같이 요청하세요.

```
> src/main.py에 TODO 주석을 3개 추가하고,  
> src/models.py에 Todo 클래스를 작성하고 TODO 주석을 2개 추가해줘
```

파일이 준비되면 MCP 검색 기능을 테스트해볼게요.

```
> 프로젝트 전체에서 TODO가 포함된 파일을 찾아줘
```

Claude가 `search_files` 도구를 사용해서 TODO가 포함된 모든 파일과 해당 줄을 찾아줄 거예요.

[스크린샷: Claude가 MCP `search_files`로 TODO를 검색한 결과를 보여주는 화면]

더 복잡한 요청도 해볼게요.

```
> src 폴더에서 함수 정의를 모두 찾아서, 각 함수의 이름과 파라미터를 정리해줘
```

이번에는 Claude가 여러 도구를 조합해요. 먼저 `list_directory`로 `src` 폴더의 파일을 확인하고, 각 파일을 `read_file`로 읽은 뒤, 함수 정의를 추출해서 정리해요.

직접 해보기

아래 요청들을 순서대로 해보면서 Claude가 어떤 MCP 도구를 사용하는지 관찰해보세요.

1. “README.md의 내용을 읽어줘”, `read_file` 사용
2. “tests 폴더에 어떤 파일이 있어?”, `list_directory` 사용
3. “.py 확장자 파일을 모두 찾아줘”, `search_files` 사용
4. “main.py의 파일 크기와 마지막 수정 시간을 알려줘”, `get_file_info` 사용

Claude가 도구를 호출할 때 어떤 도구를 선택하는지, 파라미터로 무엇을 넘기는지 주의 깊게 살펴보세요. 이것이 MCP의 작동 원리를 이해하는 핵심이에요.

MCP 도구 호출 과정 이해하기

Claude가 MCP 도구를 사용하는 과정을 단계별로 살펴볼게요.

단계	설명
1. 요청 분석	사용자의 자연어 요청을 분석해서 어떤 도구가 필요한지 판단해요
2. 도구 선택	사용 가능한 MCP 도구 중 적합한 것을 선택해요
3. 파라미터 구성	도구에 넘길 파라미터를 JSON으로 구성해요
4. 도구 실행	MCP 서버에 요청을 보내고 결과를 받아요
5. 결과 해석	도구의 반환값을 자연어로 정리해서 사용자에게 보여줘요

이 과정이 자동으로 일어나기 때문에 사용자 입장에서는 그냥 “찾아줘”라고 말하면 되는 거예요. 하지만 이 과정을 이해하고 있으면 더 정확한 요청을 할 수 있어요.

MCP 도구를 효과적으로 사용하려면 “어떤 도구가 있는지”를 알고 있는 것이 중요해요. Claude에게 “사용 가능한 도구를 보여줘”라고 물어보면 현재 연결된 모든 도구를 확인할 수 있어요.

4. MCP 도구 목록 확인 및 관리

4 등록된 MCP 서버를 관리하는 방법을 알아봐요

MCP 서버를 추가했으면, 목록을 확인하고 필요 없는 서버를 제거하는 방법도 알아야 해요.

Claude Code를 종료한 상태에서 아래 명령어를 실행해보세요.

```
claude mcp list
```

현재 등록된 모든 MCP 서버 목록이 표시돼요.

[스크린샷: claude mcp list 명령어 결과로 등록된 서버 목록이 표시되는 화면]

MCP 서버 관리에 사용하는 명령어를 정리해볼게요.

명령어	기능
<code>claude mcp list</code>	등록된 MCP 서버 목록 조회
<code>claude mcp add <이름> ...</code>	새 MCP 서버 추가
<code>claude mcp remove <이름></code>	MCP 서버 제거

직접 해보기

다른 MCP 서버도 추가해볼까요? 메모리 MCP 서버를 추가하면 Claude가 대화 간에 정보를 기억할 수 있어요.

```
claude mcp add memory -s user, npx -y @modelcontextprotocol/server-memory
```

추가 후 `claude mcp list`로 두 개의 서버가 등록되어 있는지 확인해보세요.

Claude Code를 실행하고 아래처럼 테스트해보세요.

> 내 이름은 김지은이고, 파이썬을 주로 사용해. 이걸 기억해줘

새 세션에서 다시 시작하고 물어보세요.

> 내 이름이 뭐였지?

메모리 MCP가 잘 작동하면 이전 대화에서 저장한 정보를 기억하고 있을 거예요.

MCP 생태계 탐색

파일시스템과 메모리 외에도 다양한 MCP 서버가 있어요. 커뮤니티에서 만든 서버도 많아요.

서버	기능	제공
filesystem	파일 읽기/쓰기/검색	공식
memory	영구 기억 저장	공식
brave-search	웹 검색	공식
github	GitHub 이슈/PR 관리	공식
postgres	PostgreSQL DB 쿼리	공식
slack	Slack 메시지 읽기/쓰기	커뮤니티

서버	기능	제공
notion	Notion 페이지 관리	커뮤니티

MCP 서버 목록은 <https://github.com/modelcontextprotocol/servers>에서 확인할 수 있어요. 필요한 도구가 있으면 같은 방식(`claude mcp add`)으로 추가하면 돼요.

MCP의 강력함은 “표준”에 있어요. 누구나 MCP 서버를 만들 수 있고, 같은 방식으로 Claude에 연결할 수 있어요. 마치 USB 포트에 어떤 기기든 꽂을 수 있는 것처럼요.

MCP 보안 고려사항

MCP는 편리하지만 보안에 주의해야 해요. MCP 서버에 부여한 권한은 Claude가 자유롭게 사용할 수 있거든요.

주의 사항	설명
경로 제한	파일시스템 MCP에 ~ (홈 전체)를 주지 마세요. 필요한 폴더만 지정하세요
신뢰할 수 있는 서버만	커뮤니티 MCP 서버는 소스 코드를 확인한 후 사용하세요
민감한 데이터	API 키, 비밀번호가 있는 폴더를 MCP에 노출하지 마세요
쓰기 권한	읽기만 필요한 경우 쓰기 권한을 제한하세요

5. 정리 및 다음 단계

이번 실습 핵심 정리

- MCP는 AI가 외부 도구와 통신하는 표준 프로토콜이에요
- `claude mcp add` 명령어로 MCP 서버를 등록하고, `settings.json`에 저장돼요
- 파일시스템 MCP를 통해 Claude가 파일을 읽고, 검색하고, 관리할 수 있어요
- MCP 서버는 user(전역) 또는 project(프로젝트별) 범위로 설정할 수 있어요
- 보안을 위해 최소 권한 원칙을 지켜주세요

트러블슈팅

Q. `claude mcp add` 실행 시 에러가 나요

A. Node.js 버전이 18 이상인지 확인하세요. ``node --version``으로 확인할 수 있어요. 또한 `npx`가 설치되어 있는지 ``npx --version``으로 확인하세요.

Q. MCP 서버가 연결되었는데 도구를 사용하지 못해요

A. Claude Code를 완전히 종료하고 다시 시작해보세요. MCP 설정 변경은 재시작 후 적용되는 경우가 있어요. ``/mcp`` 명령어로 현재 연결 상태를 확인할 수도 있어요.

Q. 파일시스템 MCP에서 `permission denied` 에러가 나요

A. MCP 서버에 지정한 경로에 읽기 권한이 있는지 확인하세요. 심볼릭 링크나 네트워크 드라이브는 추가 설정이 필요할 수 있어요.

Q. MCP 서버가 너무 많아서 Claude가 혼란스러워하는 것 같아요

A. 사용하지 않는 MCP 서버는 ``claude mcp remove <이름>``으로 제거하세요. 도구가 너무 많으면 Claude가 적절한 도구를 선택하는 데 방해가 될 수 있어요.

Q. `settings.json`을 직접 편집해도 되나요?

A. 네, JSON 형식만 올바르게 직접 편집해도 돼요. 하지만 JSON 문법 오류(쉼표 누락, 따옴표 불일치 등)가 있으면 Claude Code가 시작되지 않을 수 있으니 주의하세요.

실습 체크리스트

- ☐ 파일시스템 MCP 서버를 `claude mcp add`로 추가했다
- ☐ Claude Code에서 MCP 서버가 연결된 것을 확인했다
- ☐ MCP 도구로 파일을 검색하거나 읽어봤다
- ☐ `claude mcp list`로 등록된 서버 목록을 확인했다

- 여기까지 완료하면 이번 실습은 성공이에요! -

- ☐ [도전] 메모리 MCP 서버를 추가로 설치해봤다
- ☐ [도전] MCP 도구 호출 과정(도구 선택 → 파라미터 → 실행 → 결과)을 관찰했다
- ☐ [도전] settings.json의 내용을 직접 열어서 구조를 확인했다

>> 다음 차시 예고

실습 9차시: Skills 기초: 나만의 전문가

MCP가 Claude에게 도구를 연결해줬다면, Skills는 Claude에게 전문 지식을 주입해요. 나만의 스킬을 만들어서 Claude를 특정 분야의 전문가로 만들어볼 거예요.

실습 9차시

Skills 기초: 나만의 전문가

학습 목표

- ✓ Skill의 구조(이름, 설명, 트리거, 본문)를 이해하고 기본 Skill을 작성할 수 있어요
- ✓ 작성한 Skill을 Claude Code에 등록하고 실행할 수 있어요
- ✓ Skill이 정상 동작하는지 테스트하고 문제를 진단할 수 있어요

? 핵심 질문

MCP가 Claude에게 “도구”를 줬다면, 어떻게 하면 Claude에게 “전문 지식”을 줄 수 있을까요? 매번 같은 설명을 반복하지 않고, 한 번 정의해두면 자동으로 불러와지는 방법이 있을까요?

0. 오늘 만들 결과물

이번 실습이 끝나면 여러분만의 커밋 스킬이 완성돼요. `/commit`이라고 입력하면 Claude가 변경된 파일을 분석하고, 적절한 커밋 메시지를 작성하고, 확인을 거쳐 커밋까지 해줘요. 이 과정 전체가 하나의 스킬로 자동화되는 거예요.

배경지식: Skills란 무엇인가

이론편에서 MCP와 Skills의 관계를 배웠어요. MCP가 USB 포트라면, Skills는 그 포트에 연결되는 전문 앱이에요.

Skill은 Claude Code에게 특정 작업의 수행 방법을 가르치는 마크다운 파일이에요. CLAUDE.md가 “이 프로젝트는 이렇게 생겼어”라는 정보를 주는 거라면, Skill은 “이런 요청이 오면 이렇게 처리해”라는 절차를 주는 거예요.

핵심 차이를 정리하면 이래요.

- **CLAUDE.md**: 프로젝트의 맥락 정보 (수동적, 항상 로드)
- **Skills**: 특정 작업의 수행 절차 (능동적, 요청 시 로드)

준비물

- Claude Code (이전 실습에서 설정 완료)
- Git이 초기화된 프로젝트 폴더
- 텍스트 에디터

1. Skills 폴더 구조 이해

1 Skills가 어디에 저장되고 어떤 구조인지 이해해요

Skills 파일은 `.claude/commands/` 폴더 안에 마크다운 파일로 저장돼요. 두 가지 위치가 있어요.

```
~/ .claude/commands/      # 모든 프로젝트에서 사용 (개인용)
프로젝트/.claude/commands/  # 이 프로젝트에서만 사용
```

오늘은 프로젝트 폴더 안에 스킬을 만들어볼 거예요. 먼저 폴더를 만들어보세요.

실습 프로젝트로 이동해서 폴더를 만들어볼게요.

```
cd ~/practice-todo-app
git init
mkdir -p .claude/commands
```

`.claude/commands/` 안에 마크다운 파일을 만들면 그것이 곧 스킬이에요. 파일 이름이 스킬의 슬래시 명령어가 돼요.

파일 이름	실행 방법
<code>commit.md</code>	<code>/commit</code> 으로 실행
<code>review.md</code>	<code>/review</code> 로 실행
<code>test.md</code>	<code>/test</code> 로 실행
<code>deploy/staging.md</code>	<code>/deploy:staging</code> 으로 실행

폴더 안에 하위 폴더를 만들면 콜론으로 구분되는 네임스페이스가 돼요. 관련 스킬을 그룹으로 묶을 때 유용해요.

스킬의 핵심은 단순함이에요. 마크다운 파일 하나가 곧 스킬이에요. 별도의 설정 파일이나 등록 과정이 필요 없어요. 파일을 만들면 바로 사용할 수 있어요.

2. 간단한 SKILL.md 작성: 커밋 스킬

2 첫 번째 스킬을 작성해요: 커밋 자동화

텍스트 에디터로 커밋 스킬 파일을 만들어보세요.

```
code ~/practice-todo-app/.claude/commands/commit.md
```

스킬 파일의 기본 구조를 알아볼게요. 스킬은 크게 세 부분으로 구성돼요.

스킬 파일의 구조

스킬 파일은 YAML 프론트매터(선택)와 마크다운 본문으로 구성돼요.

```
---
description: "변경사항을 분석하고 커밋 메시지를 작성합니다"
---
```

커밋 스킬

아래 절차를 따라 커밋을 수행하세요.

1. ``git status``와 ``git diff``를 실행하여 변경사항을 확인하세요
2. 변경사항을 분석하여 적절한 커밋 메시지를 작성하세요
3. 커밋 메시지는 아래 형식을 따르세요:
 - 첫 줄: 타입(feat/fix/docs/refactor): 변경 요약 (50자 이내)
 - 빈 줄
 - 본문: 변경 이유와 내용 (선택)
4. 작성한 커밋 메시지를 사용자에게 보여주고 확인을 받으세요
5. 확인이 되면 ``git add``와 ``git commit``을 실행하세요

이 내용을 `commit.md` 파일에 저장하세요.

[스크린샷: VS Code에서 `commit.md` 스킬 파일을 작성하는 화면]

각 부분이 무엇을 하는지 살펴볼게요.

부분	설명
프론트매터 ---	스킬의 메타데이터예요. <code>description</code> 은 스킬 목록에서 보이는 설명이에요

부분	설명
제목 #	스킬의 이름이에요. Claude가 이 스킬을 로드할 때 참조해요
본문	Claude가 따라야 할 절차와 규칙이에요. 마크다운 형식으로 자유롭게 작성해요

커밋 스킬을 더 정교하게 만들기

기본 버전을 만들었으니 좀 더 정교한 버전으로 업그레이드해볼게요. `commit.md`를 아래 내용으로 교체하세요.

```
---
description: "변경사항을 분석하고 컨벤셔널 커밋 메시지를 작성합니다"
---
```

커밋 스킬

절차

- **변경사항 확인****
 - ``git status``로 변경된 파일 목록을 확인하세요
 - ``git diff --staged``로 스테이징된 변경사항을 확인하세요
 - 스테이징된 파일이 없으면 ``git diff``로 전체 변경사항을 확인하세요
- **변경사항 분석****
 - 각 파일의 변경 내용을 읽고 "무엇이 바뀌었는지" 파악하세요
 - 변경의 목적(새 기능, 버그 수정, 리팩토링 등)을 판단하세요
- **커밋 메시지 작성****
 - Conventional Commits 형식을 따르세요:
 - ``feat:`` 새 기능
 - ``fix:`` 버그 수정
 - ``docs:`` 문서 변경
 - ``refactor:`` 리팩토링
 - ``test:`` 테스트 추가/수정
 - ``chore:`` 빌드, 설정 변경
 - 첫 줄은 50자 이내, 한글로 작성
 - 본문이 필요하면 "왜" 변경했는지를 적으세요
- **확인 및 실행****
 - 작성한 커밋 메시지를 사용자에게 보여주세요
 - 스테이징되지 않은 파일이 있다면 어떤 파일을 포함할지 물어보세요
 - 확인을 받은 후 ``git add``와 ``git commit``을 실행하세요

규칙

- 커밋 메시지는 반드시 한글로 작성하세요
- 한 커밋에는 하나의 논리적 변경만 포함하세요
- .env, credentials 등 민감 파일은 커밋하지 마세요

좋은 스킬은 “무엇을 할지”뿐 아니라 “어떤 순서로, 어떤 규칙을 지키면서” 할지를 구체적으로 적어요. Claude는 이 절차를 따라 작업을 수행해요.

3. 트리거 설정과 테스트

3 스킬을 실행하고 동작을 확인해요

스킬 파일을 저장했으면 바로 테스트해볼 수 있어요. Claude Code를 실행하세요.

```
cd ~/practice-todo-app
claude
```

먼저 스킬이 인식되는지 확인해볼게요.

Claude Code에서 /를 입력하면 사용 가능한 스킬 목록이 표시돼요.

> /

방금 만든 /commit 스킬이 목록에 보여야 해요. description에 적은 “변경사항을 분석하고 컨벤셔널 커밋 메시지를 작성합니다”가 함께 표시될 거예요.

[스크린샷: Claude Code에서 / 입력 시 스킬 목록이 표시되고 commit 스킬이 보이는 화면]

테스트를 위해 먼저 파일을 수정해볼게요.

> src/utils.py에 날짜 포매팅 함수를 추가해줘

Claude가 파일을 수정하면, 이제 커밋 스킬을 실행해보세요.

> /commit

[스크린샷: /commit 스킬이 실행되어 변경사항을 분석하고 커밋 메시지를 제안하는 화면]

Claude가 스킬에 적합한 절차대로 움직이는 것을 볼 수 있어요.

1. git status와 git diff를 실행해서 변경사항을 확인하고
2. 변경 내용을 분석해서 타입(feats/fix/docs 등)을 판단하고
3. Conventional Commits 형식으로 커밋 메시지를 작성하고
4. 여러분에게 확인을 요청해요

직접 해보기

커밋 스킬의 동작을 더 깊이 테스트해보세요.

1. 여러 파일을 동시에 수정한 뒤 /commit을 실행해보세요. Claude가 변경사항을 어떻게 요약하나요?
2. README.md만 수정한 뒤 /commit을 실행해보세요. 타입을 docs:로 선택하나요?
3. 버그를 일부러 넣고 수정한 뒤 /commit을 실행해보세요. fix:를 선택하나요?
4. .env 파일을 만들고 수정한 뒤 /commit을 실행해보세요. 민감 파일 경고가 나오나요?

스킬에 인자 전달하기

스킬 본문에 \$ARGUMENTS를 사용하면 사용자가 입력한 추가 텍스트를 받을 수 있어요.

예를 들어 스킬 본문에 이렇게 적으면:

사용자가 제공한 추가 지시: \$ARGUMENTS

실행할 때 이렇게 입력하면:

```
> /commit WIP 커밋이니 메시지 간단하게
```

\$ARGUMENTS 자리에 “WIP 커밋이니 메시지 간단하게”가 들어가요.

\$ARGUMENTS를 활용하면 하나의 스킬로 다양한 상황을 처리할 수 있어요. 스킬을 범용적으로 만들면 서로 실행 시점에 세부 지시를 전달할 수 있는 거예요.

4. 자동 매칭 확인

4 스킬이 자동으로 매칭되는 메커니즘을 이해해요

스킬은 `/commit`처럼 직접 호출하는 것 외에도, 자연어로 요청하면 Claude가 적절한 스킬을 자동으로 선택해서 실행할 수도 있어요.

이것이 바로 description 필드의 역할이에요. Claude는 사용자의 요청과 각 스킬의 description을 비교해서 가장 적절한 스킬을 찾아요.

자동 매칭을 테스트해볼게요. 슬래시 명령어 없이 자연어로 요청해보세요.

> 지금 수정한 내용을 커밋해줘

Claude가 `/commit` 스킬을 자동으로 로드해서 실행하는지 확인해보세요.

[스크린샷: 자연어로 커밋을 요청했을 때 Claude가 자동으로 커밋 스킬을 로드하는 화면]

자동 매칭이 잘 되려면 description이 명확해야 해요. 비교해볼게요.

나쁜 description	좋은 description
“커밋”	“변경사항을 분석하고 컨벤셔널 커밋 메시지를 작성합니다”
“코드 관련 작업”	“코드 리뷰를 수행하고 개선 사항을 제안합니다”
“테스트”	“변경된 파일에 대한 단위 테스트를 작성하고 실행합니다”

description이 구체적일수록 Claude가 올바른 스킬을 선택할 확률이 높아져요.

직접 해보기

자동 매칭 실험을 해볼게요. 같은 의도를 다른 표현으로 말해보세요.

1. “변경사항 정리해서 저장해줘”, 커밋 스킬이 매칭되나요?
2. “git에 올려줘”, 커밋 스킬이 매칭되나요?
3. “코드 수정 마무리해줘”, 어떤 스킬이 매칭되나요?

매칭이 잘 안 되는 표현이 있다면, description에 동의어를 추가해보세요. 예를 들어 “변경사항을 분석하고 커밋합니다. git 저장, 코드 올리기”처럼요.

스킬 디버깅 방법

스킬이 예상대로 동작하지 않을 때 디버깅하는 방법을 알아볼게요.

1) 스킬이 목록에 안 보일 때

- 파일 위치가 `.claude/commands/` 안에 있는지 확인하세요
- 파일 확장자가 `.md`인지 확인하세요
- Claude Code를 재시작해보세요

2) 스킬이 로드되는데 절차를 안 따를 때

- 지시사항이 너무 모호하지 않은지 확인하세요
- 단계 번호를 명확히 매기세요
- “반드시”, “항상” 같은 강조 표현을 사용하세요

3) 스킬 내용을 수정했는데 반영이 안 될 때

- Claude Code를 완전히 종료하고 다시 시작하세요
- 파일이 정상적으로 저장되었는지 확인하세요

스킬은 “Claude에게 주는 지시서”예요. 지시서가 명확할수록 Claude가 정확하게 따라요. 모호한 스킬은 모호한 결과를 만들어요.

5. 정리 및 다음 단계

이번 실습 핵심 정리

- **Skills**는 `.claude/commands/` 폴더의 마크다운 파일이에요
- 파일 이름이 슬래시 명령어가 돼요 (`commit.md` → `/commit`)
- 프론트매터의 `description`은 자동 매칭에 사용돼요
- 스킬 본문에 절차, 규칙, 형식을 구체적으로 적으면 Claude가 정확하게 따라요
- `$ARGUMENTS`로 실행 시 추가 지시를 전달할 수 있어요

핵심 용어

용어	설명
Skill	Claude Code에게 특정 작업의 수행 방법을 알려주는 마크다운 파일
프론트매터	마크다운 파일 상단의 --- 로 둘러싸인 YAML 메타데이터 영역
슬래시 명령어	/로 시작하는 스킬 실행 명령어. 파일 이름에서 자동 생성
\$ARGUMENTS	스킬 실행 시 사용자가 추가로 입력한 텍스트를 받는 변수
자동 매칭	사용자의 자연어 요청을 description과 비교하여 적절한 스킬을 선택하는 기능

트러블슈팅

Q. 스킬 파일을 만들었는데 / 목록에 안 나와요

A. 파일이 ``.claude/commands/`` 폴더 안에 있는지, 확장자가 ``.md``인지 확인하세요. 폴더 이름이 정확히 ``.commands``인지도 확인하세요 (``.command``가 아닌). Claude Code를 재시작해보세요.

Q. 스킬이 실행되는데 절차를 건너뛰어요

A. 스킬 본문의 지시가 모호할 수 있어요. 각 단계를 번호로 명확히 구분하고, '반드시'라는 표현을 추가해보세요. 예: '반드시 git status를 먼저 실행하세요.'

Q. 스킬 내용을 수정했는데 이전 버전으로 동작해요

A. Claude Code를 완전히 종료(``.exit``)하고 다시 시작하세요. 실행 중에 스킬 파일을 수정하면 즉시 반영되지 않는 경우가 있어요.

Q. 자동 매칭이 잘 안 돼요. 엉뚱한 스킬이 실행돼요

A. description을 더 구체적으로 작성하세요. 또한 스킬 수가 적으면 매칭 정확도가 떨어질 수 있어요. 비슷한 스킬이 여러 개일 때는 직접 ``.스킬이름``으로 호출하는 것이 확실해요.

실습 체크리스트

- ☐ `.claude/commands/` 폴더를 만들었다
- ☐ `commit.md` 스킬 파일을 작성했다 (프론트매터 + 본문)
- ☐ `/commit`으로 스킬을 실행하고 결과를 확인했다
- ☐ 자연어로 커밋을 요청했을 때 자동 매칭이 되는지 확인했다

- 여기까지 완료하면 이번 실습은 성공이에요! -

- ☐ [도전] 스킬에 `$ARGUMENTS`를 추가하고 인자를 전달해봤다
- ☐ [도전] `description`을 수정하면서 자동 매칭 정확도 변화를 관찰했다
- ☐ [도전] 자기만의 스킬 아이디어를 1개 이상 구상했다

>> 다음 차시 예고

실습 10차시: Skills 심화: 복합 스킬

하나의 스킬으로는 부족할 때가 있어요. 여러 스킬을 조합하고, 도구 사용 권한을 제어하고, context fork로 독립 실행하는 방법을 배워볼 거예요.

실습 10차시

Skills 심화: 복합 스킬

학습 목표

- ✓ 조건 분기와 도구 호출을 포함한 복합 Skill을 구성할 수 있어요
- ✓ 기존 Skill을 조합하여 워크플로우 형태의 Skill을 설계할 수 있어요
- ✓ Skill 간 의존성과 실행 순서를 분석하여 최적 구조를 판단할 수 있어요

? 핵심 질문

지난 시간에 커밋 스킬 하나를 만들었어요. 그런데 실무에서는 “코드 리뷰 후 커밋”, “테스트 통과 후 배포”처럼 여러 작업이 연결되는 경우가 많아요. 여러 스킬을 어떻게 조합하고 관리할 수 있을까요?

0. 오늘 만들 결과물

이번 실습이 끝나면 두 개의 스킬이 완성돼요. 하나는 코드 리뷰 스킬이고, 다른 하나는 지난 시간에 만든 커밋 스킬을 업그레이드한 버전이에요. 그리고 이 둘이 연결되어 “리뷰 후 커밋”이라는 워크플로우가 완성돼요.

배경지식: 복합 스킬과 컨텍스트 관리

스킬은 단독으로도 유용하지만, 여러 스킬을 조합하면 더 강력해져요. 이때 두 가지를 고려해야 해요.

첫째, **allowed-tools**예요. 스킬이 사용할 수 있는 도구를 제한하면 안전성이 높아져요. 예를 들어 코드 리뷰 스킬은 파일을 읽기만 하면 되니까, 쓰기 도구는 제외할 수 있어요.

둘째, **context fork**예요. 스킬이 실행될 때 메인 대화와 별도의 컨텍스트에서 동작하게 하면, 스킬의 중간 과정이 메인 대화를 어지럽히지 않아요. 복잡한 분석을 수행하는 스킬에서 특히 유용해요.

준비물

- 지난 실습에서 만든 practice-todo-app 프로젝트
- `.claude/commands/commit.md` 파일 (지난 실습에서 작성)
- Claude Code

1. allowed-tools 설정

1 스킬이 사용할 수 있는 도구를 제한해요

스킬에는 `allowed-tools`를 설정할 수 있어요. 이 설정이 없으면 스킬은 Claude가 사용할 수 있는 모든 도구를 사용할 수 있어요. 안전을 위해, 스킬에 필요한 도구만 허용하는 것이 좋은 습관이에요.

지난 시간에 만든 `commit.md`를 열어보세요.

```
code ~/practice-todo-app/.claude/commands/commit.md
```

프론트매터에 `allowed-tools`를 추가해볼게요. 커밋 스킬에 필요한 도구는 Bash(git 명령어 실행)와 파일 읽기예요.

```
---
description: "변경사항을 분석하고 컨벤셔널 커밋 메시지를 작성합니다"
allowed-tools: ["Bash", "Read", "Glob", "Grep"]
---
```

커밋 스킬

(이하 기존 내용 동일)

`allowed-tools`에 명시한 도구만 이 스킬 실행 중에 사용할 수 있어요. 파일 쓰기(Write, Edit)를 제외했기 때문에, 커밋 스킬이 실행되는 동안에는 파일 내용을 수정할 수 없어요. 커밋 전에 코드를 멋대로 고치는 것을 방지하는 안전장치인 셈이에요.

도구	허용 이유
Bash	git status, git diff, git add, git commit 실행에 필요
Read	파일 내용을 읽어서 변경사항 분석에 필요
Glob	파일 목록을 조회하는 데 필요
Grep	파일 내용을 검색하는 데 필요

`allowed-tools`는 “최소 권한 원칙”의 구현이에요. 스킬이 필요한 도구만 허용하면, 의도치 않은 부작용을 예방할 수 있어요. 특히 파일을 수정하거나 외부 API를 호출하는 도구는 신중하게 허용하세요.

직접 해보기

allowed-tools의 효과를 확인해보세요.

1. 커밋 스킴을 실행한 상태에서 “이 파일 내용도 수정해줘”라고 요청해보세요
2. Write 도구가 제한되어 있기 때문에 Claude가 수정을 거부할 거예요
3. allowed-tools에 “Write”를 추가하고 같은 요청을 해보세요
4. 이번에는 수정이 가능해질 거예요

이 차이를 통해 allowed-tools의 역할을 체감할 수 있어요.

2. 두 번째 스킴 제작: 코드 리뷰

2 다른 도메인의 스킴을 만들어봐요

이번에는 코드 리뷰 스킴을 만들어볼 거예요. 이 스킴은 변경된 코드를 분석하고, 잠재적 문제와 개선 사항을 찾아주는 스킴이에요.

새 파일을 만들어보세요.

```
code ~/practice-todo-app/.claude/commands/review.md
```

아래 내용을 작성하세요.

```
---
description: "변경된 코드를 리뷰하고 개선 사항을 제안합니다"
allowed-tools: ["Bash", "Read", "Glob", "Grep"]
---
```

코드 리뷰 스킴

리뷰 절차

1. ****변경 범위 파악****
 - `git diff`를 실행하여 변경된 코드를 확인하세요
 - 변경된 파일의 전체 컨텍스트를 읽어 이해하세요
2. ****코드 품질 점검****
 - 다음 항목을 순서대로 점검하세요:
 - [] 타입 힌트가 빠진 함수가 있는가

- [] 에러 처리가 누락된 부분이 있는가
- [] 하드코딩된 값이 있는가
- [] 중복 코드가 있는가
- [] 함수가 30줄을 넘는가

3. **보안 점검**

- [] 민감 정보(API 키, 비밀번호)가 코드에 포함되어 있는가
- [] SQL 인젝션 등 입력 검증이 필요한 부분이 있는가
- [] 로그인에 민감 정보가 포함되는가

4. **결과 보고**

아래 형식으로 리뷰 결과를 정리하세요:

코드 리뷰 결과

발견된 문제 (심각도 높음)

- [파일:줄] 문제 설명

개선 제안 (선택)

- [파일:줄] 제안 내용

칭찬할 점

- 잘 작성된 부분 언급

규칙

- 리뷰는 읽기 전용이에요. 코드를 직접 수정하지 마세요
- 문제의 심각도를 높음/중간/낮음으로 구분하세요
- 개선 제안에는 항상 "왜" 개선해야 하는지 이유를 포함하세요
- 칭찬할 점을 최소 1개는 찾아주세요

\$ARGUMENTS가 있다면 해당 부분을 중점적으로 리뷰하세요.

[스크린샷: review.md 스킴 파일을 작성 완료한 화면]

이제 두 개의 스킬이 준비되었어요. 코드 리뷰 스킬을 테스트해볼게요.

```
cd ~/practice-todo-app
claude
```

파일을 하나 수정하고 리뷰 스킬을 실행해보세요.

```
> src/utls.py에 데이터를 파싱하는 함수를 추가해줘
> /review
```

[스크린샷: /review 스킬이 실행되어 코드 리뷰 결과를 보여주는 화면]

코드 리뷰 스킬이 변경사항을 분석하고, 체크리스트 항목별로 점검 결과를 보여줄 거예요. 문제가 발견되면 파일명과 줄 번호와 함께 구체적으로 알려줘요.

좋은 리뷰 스킬은 “무엇을 점검할지”를 체크리스트로 명시해요. Claude는 이 체크리스트를 하나씩 따라가면서 빠짐없이 점검해요. 체크리스트가 구체적일수록 리뷰 품질이 올라가요.

3. 자동 매칭 테스트

3 여러 스킬이 있을 때 자동 매칭이 어떻게 되는지 확인해요

이제 두 개의 스킬이 있어요. 자연어로 요청했을 때 Claude가 올바른 스킬을 선택하는지 테스트해볼 거예요.

Claude Code에서 아래 요청들을 하나씩 해보세요.

요청	기대 스킬	실제 결과
“변경사항을 커밋해줘”	/commit	직접 확인
“코드를 리뷰해줘”	/review	직접 확인
“이 코드에 문제가 있는지 봐줘”	/review	직접 확인
“수정한 내용을 저장해줘”	/commit	직접 확인
“코드 품질을 점검해줘”	/review	직접 확인

실제 결과가 기대와 다른 경우가 있을 수 있어요. 그럴 때는 description을 더 구체적으로 수정하면 정확도가 올라가요.

직접 해보기

자동 매칭이 혼동될 만한 요청을 일부러 해보세요.

1. “코드를 확인하고 커밋해줘”, 리뷰? 커밋? 둘 다?
2. “문제 없으면 커밋해줘”, 리뷰 후 커밋?
3. “이거 괜찮아 보여?”, 리뷰?

Claude가 어떤 스킬을 선택하는지 관찰하고, 왜 그 스킬을 선택했는지 물어보세요.

> 왜 그 스킬을 선택했어?

스킬 조합: 리뷰 후 커밋 워크플로우

“리뷰 후 커밋”이라는 워크플로우를 만들어볼게요. 새 스킬 파일을 만들어보세요.

```
code ~/practice-todo-app/.claude/commands/review-and-commit.md
```

```
---
```

```
description: "코드를 리뷰한 후, 문제가 없으면 커밋합니다"
```

```
allowed-tools: ["Bash", "Read", "Glob", "Grep", "Edit"]
```

```
---
```

리뷰 후 커밋

절차

1. **코드 리뷰 수행**

- `git diff`로 변경사항을 확인하세요
- 아래 항목을 점검하세요:
 - 타입 힌트 누락
 - 에러 처리 누락
 - 보안 문제
 - 코드 스타일 위반

2. **리뷰 결과 보고**

- 발견된 문제를 심각도별로 정리하세요
- 사용자에게 결과를 보여주세요

3. **분기 처리**

- **심각한 문제가 있으면**: 문제 목록을 보여주고 수정 여부를 물어보세요
- 수정을 원하면 문제를 수정하세요
- 수정을 원하지 않으면 중단하세요
- **문제가 없거나 경미하면**: 커밋 절차로 진행하세요

4. **커밋 수행**

- Conventional Commits 형식으로 커밋 메시지를 작성하세요
- 사용자 확인을 받은 후 커밋하세요

이 스킬은 리뷰와 커밋을 연결하면서, 중간에 조건 분기를 넣었어요. “문제가 있으면 수정할지 물어보고, 없으면 바로 커밋”하는 흐름이에요.

[스크린샷: /review-and-commit 스킬 실행 시 리뷰 후 커밋까지 진행되는 화면]

4. Context Fork 설정

4 스킬을 독립된 컨텍스트에서 실행하는 방법을 알아봐요

스킬이 복잡한 분석을 수행할 때, 중간 과정의 모든 출력이 메인 대화에 쌓이면 컨텍스트가 지저분해져요. context fork를 사용하면 스킬이 별도의 컨텍스트에서 실행되고, 최종 결과만 메인 대화로 돌아와요.

이것은 마치 팀원에게 일을 위임하는 것과 비슷해요. “이거 분석해서 결과만 알려줘”라고 하면, 팀원이 중간에 뭘 했는지는 모르지만 결론만 깔끔하게 받아볼 수 있잖아요.

코드 리뷰 스킬에 context fork를 적용해볼게요. review.md를 수정하세요.

```
---
description: "변경된 코드를 리뷰하고 개선 사항을 제안합니다"
allowed-tools: ["Bash", "Read", "Glob", "Grep"]
---
```

코드 리뷰 스킬

(기존 내용은 유지하되, 맨 아래에 아래 내용 추가)

실행 모드

이 스킬은 서브에이전트로 실행되어야 해요. 중간 분석 과정은 사용자에게 보여주지 않고, 최종 리뷰 결과만 깔끔하게 정리해서 전달하세요.

Claude Code에서 스킬을 서브에이전트 모드로 실행하면 독립된 컨텍스트에서 작업이 수행돼요. 현재 Claude Code에서는 스킬 실행 시 자동으로 이 모드를 적용할 수 있어요.

직접 해보기

context fork의 효과를 비교해보세요.

1. /review를 실행하고 Claude가 중간 과정을 어떻게 보여주는지 관찰하세요
2. 리뷰가 끝나면 메인 대화에 얼마나 많은 내용이 쌓여 있는지 확인하세요

중간 과정이 많은 복잡한 스킬일수록 context fork의 효과가 커져요. 특히 여러 파일을 분석하는 스킬에서는 중간에 파일 내용이 계속 출력되면서 대화가 길어지거든요.

스킬 설계 패턴 정리

지금까지 배운 스킬 설계 패턴을 정리해볼게요.

패턴	설명	예시
단일 작업	하나의 명확한 작업을 수행	커밋, 리뷰, 테스트
파이프라인	여러 단계를 순서대로 수행	리뷰 → 커밋
조건 분기	결과에 따라 다른 경로 실행	문제 있으면 수정, 없으면 진행
도구 제한	allowed-tools로 안전 범위 설정	리뷰: 읽기만, 커밋: 읽기+실행
인자 활용	\$ARGUMENTS로 유연하게	특정 파일만 리뷰

복합 스킬을 설계할 때 가장 중요한 원칙은 “각 스킬은 하나의 책임만”이에요. 커밋 스킬은 커밋만, 리뷰 스킬은 리뷰만 담당하고, 이를 조합하는 별도의 스킬을 만드는 것이 유지보수에 유리해요.

실무에서 유용한 스킬 아이디어

여러분의 업무에 적용할 수 있는 스킬 아이디어를 몇 가지 소개할게요.

스킬 이름	용도
/deploy	배포 절차 자동화 (테스트 → 빌드 → 배포 → 확인)
/doc	코드에서 문서를 자동 생성 (함수별 설명, API 문서)
/refactor	코드 리팩토링 (중복 제거, 함수 분리, 네이밍 개선)
/translate	코드 주석/문서를 다른 언어로 번역
/migrate	라이브러리 버전 업그레이드 가이드

스킬 이름	용도
/daily	일일 업무 보고서 자동 작성 (git log 기반)

5. 정리 및 다음 단계

이번 실습 핵심 정리

- **allowed-tools**로 스킬이 사용할 수 있는 도구를 제한하여 안전성을 높여요
- 서로 다른 도메인의 스킬을 만들고, 파이프라인으로 조합할 수 있어요
- 자동 매칭은 description의 구체성에 비례해요
- **context fork**로 복잡한 스킬의 중간 과정을 숨기고 결과만 받을 수 있어요
- 좋은 스킬 설계의 원칙: 하나의 스킬, 하나의 책임

트러블슈팅

Q. allowed-tools를 설정했는데 Claude가 다른 도구도 사용해요

A. 프론트매터의 YAML 형식이 올바른지 확인하세요. 배열은 대괄호와 쌍따옴표를 사용해야 해요: ``["Bash", "Read"]``. 오타가 있으면 설정이 무시될 수 있어요.

Q. 리뷰 스킬이 코드를 수정하려고 해요

A. allowed-tools에서 Write와 Edit를 제거했는지 확인하세요. 또한 스킬 본문에 '코드를 직접 수정하지 마세요'라는 규칙을 명시적으로 적어주세요.

Q. 스킬이 세 개인데 자동 매칭이 자꾸 헛갈려요

A. 각 스킬의 description이 서로 겹치지 않도록 작성하세요. 커밋은 '저장/커밋/git', 리뷰는 '점검/리뷰/품질', 조합은 '리뷰 후 커밋'처럼 키워드를 구분하세요.

Q. review-and-commit 스킬이 리뷰만 하고 커밋은 안 해요

A. 스킬 본문의 분기 조건을 확인하세요. '문제가 있으면 중단'이라는 조건 때문에 매번 중단될 수 있어요. 경미한 문제는 경고만 하고 진행하도록 조건을 조정해보세요.

Q. context fork가 잘 적용되는지 모르겠어요

A. 현재 Claude Code 버전에서는 스킬의 서브에이전트 실행이 자동으로 처리되는 경우가 많아요. 차이를 확인하려면 복잡한 분석(10개 이상 파일 리뷰)을 시키면서 중간 출력이 얼마나 보이는지 비교해보세요.

실습 체크리스트

- ☐ 커밋 스킬에 allowed-tools를 설정했다
- ☐ 코드 리뷰 스킬을 새로 작성했다
- ☐ 자연어 요청으로 올바른 스킬이 매칭되는지 테스트했다
- ☐ 리뷰+커밋 조합 스킬을 만들어봤다

- 여기까지 완료하면 이번 실습은 성공이에요! -

- ☐ [도전] allowed-tools를 변경하면서 도구 제한 효과를 확인했다
- ☐ [도전] context fork 설정을 적용해봤다
- ☐ [도전] 자기 업무에 맞는 스킬 아이디어를 2개 이상 구상했다

>> 다음 차시 예고

실습 11차시: AI에게 큰 작업 시키기

스킬 하나로 해결할 수 없는 대규모 작업이 있어요. 멀티파일 수정, 자동 테스트, 실패 시 재시도, 에이전트 루프의 진짜 위력을 체험해볼 거예요.

실습 11차시

AI에게 큰 작업 시키기

학습 목표

- ✓ 복수의 도구와 긴 컨텍스트를 다루는 대규모 에이전트 태스크를 실행할 수 있어요
- ✓ 에이전트 실행 로그를 분석하여 병목 구간을 식별할 수 있어요
- ✓ 에이전트의 자율성 수준(허용/제한)을 상황에 맞게 조절할 수 있어요

? 핵심 질문

AI에게 “이 프로젝트 전체를 리팩토링해줘”라고 시키면 어떤 일이 벌어질까요? 파일 10개를 동시에 수정하고, 테스트가 깨지면 알아서 고치는, 그런 게 가능할까요?

0. 오늘 만들 결과물

이번 실습이 끝나면 여러분은 Claude에게 “멀티파일 리팩토링”이라는 대규모 작업을 시키고, Claude가 분석 → 수정 → 테스트 → 자동 수정의 루프를 돌면서 작업을 완료하는 과정을 직접 관찰하게 돼요. 에이전트 루프가 실제로 어떻게 작동하는지 눈으로 보는 거예요.

배경지식: 에이전트 루프

이론편에서 에이전트 루프(인식 → 계획 → 실행 → 관찰)를 배웠어요. Claude Code는 이 루프를 자동으로 수행하는 에이전트예요.

작은 작업에서는 루프가 한두 번이면 끝나요. “함수 하나 작성해줘”라고 하면 한 번의 루프로 충분하죠. 하지만 큰 작업에서는 루프가 수십 번 반복돼요. 파일을 분석하고, 수정하고, 테스트하고, 실패하면 다시 분석하고, 이 과정이 자동으로 돌아가요.

이 루프를 더 구체적으로 표현하면 **Gather → Act → Verify** 세 단계예요.

- **Gather**: 정보 수집 (파일 읽기, 구조 분석)
- **Act**: 행동 수행 (코드 수정, 파일 생성)
- **Verify**: 결과 검증 (테스트 실행, 빌드 확인)

준비물

- practice-todo-app 프로젝트 (이전 실습에서 사용)
- Claude Code
- Python 3.11+ 및 pytest (테스트 실행용)

1. 프로젝트 분석 요청

1 Claude에게 프로젝트 전체를 분석시켜요

먼저 Claude가 프로젝트 전체를 이해하게 해야 해요. 큰 작업을 시키기 전에 “전체 그림”을 파악하는 단계가 필요하거든요. 이것이 Gather 단계예요.

프로젝트를 좀 더 실습에 적합하게 만들어볼게요. Claude Code를 실행하고 아래처럼 요청하세요.

```
cd ~/practice-todo-app
claude

> 이 프로젝트를 간단한 할 일 관리 CLI 앱으로 만들어줘.
> src/main.py에 CLI, src/models.py에 Todo 모델,
> src/utls.py에 유틸리티 함수를 작성하고,
> tests/test_main.py에 기본 테스트도 작성해줘.
> 일부러 개선할 여지를 남겨줘 (타입 힌트 누락, 에러 처리 부족 등).
```

[스크린샷: Claude가 프로젝트 전체 파일을 생성하는 화면]

Claude가 기본 코드를 생성하면, 이제 전체 분석을 요청해볼게요.

- > 이 프로젝트 전체를 분석해줘.
- > 각 파일의 역할, 파일 간 의존 관계, 잠재적 문제를 정리해줘.

Claude가 분석을 수행하는 과정을 관찰해보세요. Claude는 아래 순서로 움직여요.

순서	도구	행동
1	Glob/Bash	프로젝트의 파일 목록을 조회해요
2	Read	각 파일의 내용을 읽어요
3	Grep	import 문을 검색해서 의존 관계를 파악해요
4	-	수집한 정보를 종합해서 분석 결과를 정리해요

이 과정이 에이전트 루프의 Gather 단계예요. Claude가 행동하기 전에 충분한 정보를 수집하는 거예요.

큰 작업을 시키기 전에 “먼저 전체를 분석해줘”라고 요청하는 것이 중요해요. 분석 없이 바로 수정을 시키면 Claude가 전체 맥락을 놓치고 부분적으로만 수정할 수 있거든요.

직접 해보기

Claude의 분석 결과를 검토해보세요.

1. Claude가 찾아낸 문제 목록이 정확한가요?
2. 의존 관계 분석이 맞나요? (어떤 파일이 어떤 파일을 import 하는지)
3. 놓친 문제가 있나요?

분석 결과에 빠진 부분이 있다면 추가로 알려주세요. “에러 처리 부분도 점검해줘”처럼 구체적으로 요청하면 더 깊이 분석해요.

2. 멀티파일 수정 지시

2 여러 파일을 동시에 수정하는 큰 작업을 시켜요

이제 분석 결과를 바탕으로 멀티파일 수정을 요청해볼 거예요. 이것이 Act 단계예요.

아래처럼 요청해보세요.

- > 분석 결과를 바탕으로 아래 개선을 수행해줘:
- > 1. 모든 함수에 타입 힌트 추가
- > 2. 모든 함수에 Google 스타일 docstring 추가 (한글)
- > 3. 에러 처리 보강 (구체적 예외 타입 사용)
- > 4. 테스트 커버리지 확대 (각 함수당 최소 1개 테스트)
- > 5. 수정 후 모든 테스트가 통과하는지 확인

[스크린샷: Claude가 여러 파일을 동시에 수정하는 화면]

이 요청은 단순하지 않아요. Claude는 여러 파일을 돌아다니면서 수정해야 하고, 파일 간 의존성도 고려해야 해요. 예를 들어 `models.py`의 클래스를 수정하면 `main.py`에서 그 클래스를 사용하는 부분도 맞춰서 바꿔야 하죠.

Claude가 수정하는 과정을 관찰해보세요. 보통 이런 순서로 진행돼요.

순서	행동
1	의존성 순서를 파악해요 (models → utils → main)
2	가장 기초가 되는 파일부터 수정해요 (models.py)
3	상위 파일을 수정하면서 하위 변경에 맞춰 조정해요
4	테스트 파일을 수정해요
5	테스트를 실행해서 확인해요

멀티파일 수정에서 핵심은 “순서”예요. Claude는 파일 간 의존성을 분석해서, 기초가 되는 파일부터 수정하고 상위로 올라가요. 이 순서가 뒤바뀌면 중간에 에러가 발생할 수 있어요.

대규모 작업 요청의 좋은 패턴

큰 작업을 시킬 때는 요청의 구조가 중요해요.

나쁜 요청	좋은 요청
“코드 좀 개선해줘”	“모든 함수에 타입 힌트를 추가하고 테스트를 확인해줘”
“전체적으로 고쳐줘”	“1. 타입 힌트 2. docstring 3. 에러 처리 순으로 개선해줘”
“완벽하게 만들어줘”	“CLAUDE.md 규칙을 기준으로 위반 사항을 수정해줘”

좋은 요청의 특징은 구체적이고, 번호가 있고, 완료 조건이 명확한 것이예요.

3. 테스트 실행과 자동 수정 관찰

3 테스트 실패 시 Claude가 자동으로 수정하는 과정을 관찰해요

멀티파일 수정 후 테스트를 실행하면, 높은 확률로 일부 테스트가 실패해요. 이때 Claude가 어떻게 반응하는지가 에이전트의 진가가 드러나는 순간이예요.

Claude가 테스트를 실행하는 것을 관찰해보세요. CLAUDE.md에 테스트 명령어를 적어두었기 때문에 Claude는 `pytest tests/ -v`를 자동으로 실행할 거예요.

테스트가 실패하면 Claude는 이렇게 움직여요.

루프	단계	행동
1회차	Verify	pytest 실행 → 3개 중 1개 실패
1회차	Gather	실패한 테스트의 에러 메시지 분석
1회차	Act	원인 파악 후 코드 수정
2회차	Verify	pytest 재실행 → 전부 통과

이 과정을 화면에서 직접 관찰하세요. Claude가 에러 메시지를 읽고, 원인을 분석하고, 코드를 수정하고, 다시 테스트를 돌리는 전체 루프를 볼 수 있어요.

[스크린샷: 테스트 실패 후 Claude가 자동으로 원인을 분석하고 수정하는 화면]

직접 해보기

자동 수정 루프를 더 깊이 관찰해보세요.

1. Claude가 테스트를 몇 번 반복 실행하는지 세어보세요
2. 각 루프에서 Claude가 어떤 도구를 사용하는지 기록해보세요
3. Claude가 “원인”을 어떻게 추론하는지 관찰하세요 (에러 메시지에서 어떤 부분을 읽나요?)

만약 Claude가 3번 이상 반복해도 테스트를 통과하지 못한다면, 그것은 Claude에게 추가 힌트가 필요하다는 신호예요. “에러가 파라미터 타입 때문인 것 같아”처럼 방향을 잡아주세요.

테스트를 일부러 실패시켜보기

자동 수정 루프를 더 명확하게 관찰하기 위해, 일부러 테스트를 실패시켜볼게요.

> `tests/test_main.py`에 일부러 실패하는 테스트를 2개 추가해줘.

> 하나는 존재하지 않는 함수를 호출하고,

- > 다른 하나는 잘못된 반환값을 기대하는 테스트야.
- > 그런 다음 모든 테스트가 통과하도록 수정해줘.

Claude가 이 요청을 처리하는 과정을 지켜보세요.

1. 먼저 실패하는 테스트를 추가해요
2. pytest를 실행해서 실패를 확인해요
3. 실패 원인을 분석해요 (함수가 없다, 반환값이 다르다)
4. 원인에 맞게 코드를 수정해요 (함수 추가, 로직 수정)
5. 다시 pytest를 실행해요
6. 모두 통과하면 완료를 보고해요

이 과정 전체가 자동이에요. 여러분은 지켜보기만 하면 돼요.

에이전트 루프의 핵심 가치는 “실패를 자동으로 복구하는 능력”이에요. 사람이라면 에러를 보고 → 원인을 분석하고 → 수정하고 → 다시 실행하는 이 과정을 수동으로 해야 하지만, 에이전트는 이것을 자동으로 반복해요.

4. 에이전트 루프(Gather/Act/Verify) 확인

4 에이전트 루프의 세 단계를 명확하게 구분해봐요

지금까지 관찰한 Claude의 행동을 Gather/Act/Verify 프레임워크로 정리해볼게요. 큰 작업을 Claude에게 맡길 때, 이 세 단계를 의식하면 더 효과적으로 지시할 수 있어요.

Gather (정보 수집)

Claude가 행동하기 전에 필요한 정보를 모으는 단계예요.

- 파일 구조 파악 (Glob, Bash)
- 파일 내용 읽기 (Read)
- 패턴 검색 (Grep)
- 의존성 분석 (import문 추적)
- 기존 테스트 확인

여러분이 할 수 있는 것: “먼저 전체를 분석해줘”라고 명시적으로 Gather를 요청하세요.

Act (행동 수행)

수집한 정보를 바탕으로 실제 변경을 수행하는 단계예요.

- 코드 수정 (Edit, Write)
- 파일 생성/삭제
- 설정 변경

여러분이 할 수 있는 것: 변경 범위와 우선순위를 명확히 지시하세요.

Verify (결과 검증)

변경이 올바른지 확인하는 단계예요.

- 테스트 실행 (Bash: pytest)
- 빌드 확인 (Bash: ruff check)
- 결과 보고

여러분이 할 수 있는 것: CLAUDE.md에 검증 명령어를 적어두세요. Claude가 자동으로 사용해요.

단계	Claude가 하는 일	여러분이 도울 수 있는 것
Gather	파일 읽기, 구조 분석	CLAUDE.md에 프로젝트 구조 작성
Act	코드 수정, 파일 생성	명확한 요구사항과 우선순위
Verify	테스트, 빌드 확인	CLAUDE.md에 테스트/빌드 명령어

직접 해보기

에이전트 루프를 의식적으로 분리해서 사용해 보세요.

1단계: “프로젝트 전체를 분석하고 개선 포인트를 정리해줘” (Gather만)

분석 결과를 확인한 뒤,

2단계: “분석 결과 중 1, 3, 5번을 수정해줘” (Act만)

수정 결과를 확인한 뒤,

3단계: “모든 테스트를 실행하고 결과를 알려줘” (Verify만)

이렇게 단계를 나누면 각 단계에서 Claude의 행동을 더 정밀하게 제어할 수 있어요.

에이전트 자율성 조절

Claude에게 얼마나 자유를 줄 것인지는 상황에 따라 달라요.

수준	설명	적합한 상황
높은 자율성	분석부터 수정, 테스트까지 한번에	잘 정의된 반복 작업, 테스트가 충분한 프로젝트
중간 자율성	수정은 하되 매 단계 확인	새로운 기능 추가, 중요한 리팩토링
낮은 자율성	분석만 하고 수정은 승인 후	프로덕션 코드, 보안 관련 변경

자율성을 높이려면 Claude Code를 `--dangerously-skip-permissions` 모드로 실행할 수도 있지만, 일반적인 실습에서는 기본 모드로 충분해요. Claude가 파일을 수정하거나 명령어를 실행할 때 확인을 요청하는 것이 안전해요.

에이전트의 자율성은 “테스트 커버리지”에 비례해서 높여야 해요. 테스트가 촘촘하면 Claude가 잘못 수정해도 테스트에서 잡히니까요. 테스트가 없는 코드에서는 자율성을 낮추세요.

5. 정리 및 다음 단계

이번 실습 핵심 정리

- 큰 작업은 **Gather → Act → Verify** 세 단계로 구성돼요
- 멀티파일 수정 시 Claude는 의존성 순서를 파악해서 기초 파일부터 수정해요
- 테스트 실패 시 Claude는 자동으로 원인 분석 → 수정 → 재테스트 루프를 돌아요
- 에이전트의 자율성은 테스트 커버리지에 비례해서 조절해야 해요
- CLAUDE.md에 빌드/테스트 명령어를 적어두면 Verify 단계가 자동화돼요

핵심 용어

용어	설명
에이전트 루프	인식→계획→실행→관찰의 반복 사이클. Gather/Act/Verify로도 표현
Gather	행동 전 정보를 수집하는 단계. 파일 읽기, 구조 분석 등
Act	수집한 정보를 바탕으로 실제 변경을 수행하는 단계

용어	설명
Verify	변경 결과를 검증하는 단계. 테스트 실행, 빌드 확인 등
멀티파일 수정	여러 파일을 동시에 수정하는 대규모 작업
자동 수정 루프	테스트 실패 시 원인 분석→수정→재테스트를 자동 반복하는 과정

트러블슈팅

Q. Claude가 수정을 시작하지 않고 분석만 계속해요

A. 분석이 끝났으면 '이제 수정을 시작해줘'라고 명시적으로 Act 단계로 전환을 요청하세요. Claude는 불확실하면 행동보다 분석을 선호하는 경향이 있어요.

Q. 테스트가 실패하는데 Claude가 자동으로 수정하지 않아요

A. Claude에게 '테스트가 실패하면 원인을 분석하고 자동으로 수정한 뒤 다시 테스트를 실행해줘'라고 명시적으로 지시하세요. 또는 CLAUDE.md에 '수정 후 반드시 테스트를 실행하고, 실패 시 수정을 반복한다'는 규칙을 추가하세요.

Q. Claude가 자동 수정 루프에서 같은 실수를 반복해요

A. Claude에게 힌트를 주세요. '이전 수정이 같은 에러를 반복하고 있어. 다른 접근 방식을 시도해봐'라고 하면 전략을 바꿔요. 3회 이상 같은 에러면 사람이 방향을 잡아주는 것이 효율적이에요.

Q. 멀티파일 수정 중에 Claude가 엉뚱한 파일을 수정해요

A. 수정 대상 파일을 명시적으로 지정하세요. '수정할 파일: src/main.py, src/models.py, tests/test_main.py. 다른 파일은 건드리지 마'라고 범위를 제한하세요.

실습 체크리스트

- ☐ Claude에게 프로젝트 전체 분석을 요청하고 결과를 확인했다

- ☐ 멀티파일 수정을 요청하고 Claude의 수정 순서를 관찰했다
- ☐ 테스트 실패 후 Claude가 자동으로 수정하는 과정을 관찰했다
- ☐ Gather/Act/Verify 세 단계를 구분할 수 있다

- 여기까지 완료하면 이번 실습은 성공이에요! -

- ☐ [도전] 에이전트 루프 횟수를 세고 각 루프의 도구 사용을 기록했다
- ☐ [도전] Gather/Act/Verify를 분리해서 단계별로 요청해봤다
- ☐ [도전] 자율성 수준을 바꿔가며 결과 차이를 비교했다

>> 다음 차시 예고

실습 12차시: 나만의 AI 워크플로우 설계

지금까지 배운 프롬프트, CLAUDE.md, MCP, Skills, 에이전트 루프를 모두 조합해서 여러분의 실제 업무를 자동화하는 워크플로우를 설계해볼 거예요. 실습편의 피날레예요.

실습 12차시

나만의 AI 워크플로우 설계

학습 목표

- ✓ 자기 업무에 최적화된 AI 워크플로우를 설계할 수 있어요
- ✓ 프롬프트 + 컨텍스트 + MCP + Skills를 조합한 자동화를 구축할 수 있어요
- ✓ 구축한 워크플로우의 효과를 측정하고 개선 방향을 제안할 수 있어요

? 핵심 질문

지금까지 프롬프트, CLAUDE.md, MCP, Skills, 에이전트 루프를 배웠어요. 이 모든 것을 조합하면 여러분의 업무에서 어떤 것이 자동화될 수 있을까요? 그리고 자동화하면 안 되는 것은 무엇일까요?

0. 오늘 만들 결과물

이번 실습은 실습편 전체의 피날레예요. 지금까지 배운 모든 기술을 조합해서 여러분의 실제 반복 업무 1개를 AI로 자동화하는 워크플로우를 설계하고, 실행하고, 평가하고, 문서화하는 것이 목표예요. 이번 실습이 끝나면 여러분만의 AI 워크플로우 문서가 완성돼요.

배경지식: 워크플로우 설계의 원칙

AI 워크플로우는 단순히 “AI에게 일을 시키는 것”이 아니예요. 좋은 워크플로우는 세 가지 질문에 답할 수 있어야 해요.

1. **무엇을 자동화하는가:** 반복적이고, 규칙이 명확하고, 실수 비용이 낮은 작업이 적합해요
2. **어떤 도구를 조합하는가:** 프롬프트, CLAUDE.md, MCP, Skills 중 필요한 것을 선택해요
3. **어떻게 검증하는가:** 자동화된 결과가 올바른지 확인하는 방법이 있어야 해요

이 세 질문에 답하지 못하는 자동화는 오히려 위험해요. “AI가 잘못했는데 그걸 몰랐다”는 상황이 가장 나쁜 시나리오거든요.

준비물

- 지금까지의 실습 환경 (Claude Code + MCP + Skills)
- 자동화하고 싶은 반복 업무 1개 (아이디어만 가져오세요)
- 텍스트 에디터

1. 반복 업무 1개 선택

1 자동화할 업무를 선택하고 분석해요

먼저 여러분의 업무 중에서 자동화할 대상을 하나 선택해야 해요. 좋은 대상을 고르는 것이 워크플로우 성공의 절반이에요.

아래 기준으로 업무를 평가해보세요.

좋은 자동화 대상의 조건

기준	설명	가중치
반복 빈도	주 1회 이상 반복되는 작업	높음
규칙 명확성	절차가 명확하고 예외가 적은 작업	높음
실수 비용	자동화 실패 시 피해가 적은 작업	높음
소요 시간	수동으로 하면 30분 이상 걸리는 작업	중간
텍스트 기반	입출력이 텍스트인 작업 (AI가 잘하는 영역)	중간

업무 후보 예시

직종별로 자동화할 수 있는 업무 예시를 몇 가지 소개할게요.

직종	자동화 후보 업무
개발자	코드 리뷰 + 커밋, 릴리스 노트 작성, API 문서 생성
기획자	회의록 정리 → 액션 아이템 추출, 요구사항 문서 작성
마케터	주간 리포트 요약, SNS 콘텐츠 초안 작성
데이터 분석가	데이터 전처리 + EDA 리포트 생성
교육자	수업 자료 초안 작성, 퀴즈 문항 생성
연구자	논문 요약 + 관련 연구 정리

직접 해보기

아래 워크시트를 채워보세요.

자동화 대상 업무: _____

반복 빈도: 일 / 주 / 월 ____ 회

현재 소요 시간: ____ 분

절차 (3~7단계로 정리):

1. _____
2. _____
3. _____
4. _____
5. _____

실패 시 피해: 높음 / 중간 / 낮음

이 업무를 선택한 이유: _____

자동화 대상을 고를 때 “가장 어려운 업무”가 아니라 “가장 반복적이고 지루한 업무”를 고르세요. AI 자동화의 1순위 가치는 시간 절약이지, 창의적 문제 해결이 아니에요.

자동화하면 안 되는 것

모든 업무가 자동화에 적합한 것은 아니에요. 아래 유형은 주의가 필요해요.

유형	이유
최종 의사결정	인사 평가, 투자 결정 등은 AI가 보조할 수 있지만 최종 결정은 사람이
법적 책임이 있는 문서	계약서, 법률 문서 등은 AI 초안 후 반드시 전문가 검토
민감한 개인정보 처리	개인정보를 AI에 입력하면 보안 문제 발생 가능
검증이 어려운 작업	자동화 결과의 옳고 그름을 사람이 판단하기 어려운 작업

2. 프롬프트 + 컨텍스트 + 도구 설계

2 워크플로우에 필요한 구성 요소를 설계해요

업무를 선택했으면 이제 구성 요소를 설계할 차례예요. 지금까지 배운 도구들 중에서 어떤 것이 필요한지 골라야 해요.

아래 설계 프레임워크를 사용하세요.

설계 프레임워크

구성 요소	질문	도구
프롬프트	AI에게 어떤 역할과 작업을 부여할까?	프롬프트 구조화 (실습 1 3)
컨텍스트	AI가 알아야 할 배경 정보는?	CLAUDE.md (실습 7)
도구	AI가 접근해야 할 외부 리소스는?	MCP (실습 8)
절차	작업을 어떤 순서로 수행할까?	Skills (실습 9 10)
검증	결과가 올바른지 어떻게 확인할까?	에이전트 루프 (실습 11)

예시: “주간 코드 리뷰 리포트” 워크플로우 설계

구체적인 예시로 설계 과정을 보여줄게요.

업무: 매주 금요일, 이번 주 커밋을 분석해서 팀 코드 리뷰 리포트를 작성하는 업무

프롬프트 설계

Role: 시니어 개발자로서 코드 리뷰 보고서를 작성하세요

Context: 이번 주 (월~금) 커밋 기반, 팀 프로젝트

Task: 주요 변경사항 요약, 코드 품질 이슈, 개선 제안 도출

Format: 마크다운 리포트, 팀원별 섹션 구분

컨텍스트 설계 (CLAUDE.md)

주간 리뷰 규칙

- 리포트는 한글로 작성
- 팀 코딩 컨벤션: PEP 8 + 타입 힌트 필수
- 리뷰 관점: 가독성, 유지보수성, 보안
- 칭찬: 지적 비율 = 2:1 이상

도구 설계 (MCP)

- 파일시스템 MCP: 프로젝트 코드 접근

- **GitHub MCP (선택):** PR 정보 접근

절차 설계 (Skill)

description: "이번 주 커밋을 분석하여 주간 코드 리뷰 리포트를 생성합니다"

주간 코드 리뷰 리포트

1. `git log --since="last monday" --oneline`로 이번 주 커밋 목록 확인
2. 각 커밋의 diff를 분석하여 주요 변경사항 파악
3. 팀원별로 변경사항 그룹화
4. 코드 품질 이슈 식별 (타입 힌트, 에러 처리, 테스트)
5. 개선 제안 작성
6. docs/weekly-review-YYYY-MM-DD.md에 리포트 저장

검증 설계

- 리포트에 모든 팀원이 포함되었는지 확인
- 칭찬:지적 비율이 2:1 이상인지 확인
- 누락된 커밋이 없는지 git log와 대조

직접 해보기

여러분이 선택한 업무에 대해 같은 형식으로 설계해보세요.

프롬프트: Role / Context / Task / Format

- Role: _____
- Context: _____
- Task: _____
- Format: _____

컨텍스트 (CLAUDE.md에 추가할 내용):

- _____
- _____

도구 (필요한 MCP):

- _____

절차 (Skill 본문):

1. _____
2. _____
3. _____

검증 방법:

- _____

3. 실제 실행과 결과 평가

3 설계한 워크플로우를 실제로 실행하고 평가해요

설계가 끝났으면 실제로 구현하고 실행해볼 차례예요. 설계와 현실 사이에는 항상 간극이 있어요. 이 간극을 발견하고 메우는 것이 이번 단계의 목표예요.

아래 순서로 진행하세요.

1) 파일 준비

```
mkdir -p ~/my-workflow/.claude/commands  
cd ~/my-workflow
```

CLAUDE.md를 만들어서 설계한 컨텍스트를 적으세요. Skill 파일도 만들어서 설계한 절차를 적으세요.

2) 첫 실행

Claude Code를 실행하고 워크플로우 스킬을 호출하세요.

```
claude  
> /내가만든스킬이름
```

3) 관찰과 기록

첫 실행에서 아래 항목을 기록하세요.

관찰 항목	기록
소요 시간	워크플로우 실행에 몇 분 걸렸는지
성공 여부	의도한 결과물이 나왔는지
빠진 단계	설계에는 있었지만 실행에서 빠진 단계
예상 못한 문제	설계 시 생각하지 못한 문제

관찰 항목	기록
품질 수준	결과물의 품질 (1 5점)

4) 평가 기준

워크플로우의 효과를 평가하는 기준은 세 가지예요.

기준	측정 방법	목표
시간 절약	수동 소요 시간 vs AI 소요 시간	50% 이상 절감
품질	결과물을 수동으로 만든 것과 비교	80% 이상 일치
재현성	같은 입력에 같은 품질의 결과가 나오는지	3번 중 3번 성공

직접 해보기

워크플로우를 3번 실행해보세요. 매번 결과가 비슷한가요?

만약 결과가 들쭉날쭉하다면, 다음을 점검하세요.

- 프롬프트가 너무 모호하지 않은지
- CLAUDE.md에 규칙이 충분히 구체적인지
- Skill의 절차가 빈틈없이 정의되어 있는지

3번 실행 결과를 비교해서, 일관성이 떨어지는 부분을 찾아 개선해보세요.

워크플로우의 가치는 “한 번 잘 되는 것”이 아니라 “매번 안정적으로 되는 것”이에요. 재현성이 낮으면 자동화의 의미가 없어요. 재현성을 높이려면 스킬의 절차를 더 구체적으로 작성하세요.

4. 워크플로우 문서 작성

4

워크플로우를 문서화해서 누구나 사용할 수 있게 만들어요

워크플로우가 동작하면 이제 문서로 정리할 차례예요. 잘 정리된 워크플로우 문서는 다른 사람(또는 미래의 나)이 바로 사용할 수 있게 해줘요.

프로젝트 폴더에 워크플로우 문서를 만들어보세요.

```
code ~/my-workflow/WORKFLOW.md
```

아래 템플릿을 사용해서 문서를 작성하세요.

【워크플로우 이름】

개요

- 목적: [이 워크플로우가 해결하는 문제]
- 대상 업무: [자동화하는 업무]
- 예상 소요 시간: [수동 대비 AI 소요 시간]

사전 준비

- [] Claude Code 설치 확인
- [] MCP 서버 등록: [필요한 MCP 목록]
- [] CLAUDE.md 작성 완료
- [] Skill 파일 작성 완료

실행 방법

1. `cd [프로젝트 경로]`
2. `claude`
3. `[스킬 이름]` 또는 "[자연어 요청]"

구성 요소

CLAUDE.md

[핵심 내용 요약]

Skill

[파일 경로와 핵심 절차 요약]

MCP

[사용하는 MCP 서버와 역할]

결과물

- 형식: [마크다운/JSON/코드 등]
- 위치: [저장 경로]
- 검증 방법: [어떻게 확인하는지]

알려진 한계

- [한계 1]
- [한계 2]

개선 계획

- [] [향후 개선할 점 1]
- [] [향후 개선할 점 2]

[스크린샷: 완성된 WORKFLOW.md 문서의 화면]

직접 해보기

워크플로우 문서를 작성한 후 아래를 확인해보세요.

1. 문서만 보고 다른 사람이 워크플로우를 실행할 수 있나요?
2. 사전 준비 체크리스트가 빠짐없이 정리되어 있나요?
3. “알려진 한계”에 솔직하게 적었나요?

Claude에게 문서를 보여주고 피드백을 받아보는 것도 좋은 방법이에요.

> WORKFLOW.md를 읽고, 빠진 내용이나 모호한 부분을 지적해줘

5. 전체 정리: 실습편 총 복습

이번 차시로 실습편 전체가 마무리돼요. 12개 차시에서 배운 내용을 하나의 흐름으로 정리해볼게요.

실습편 로드맵

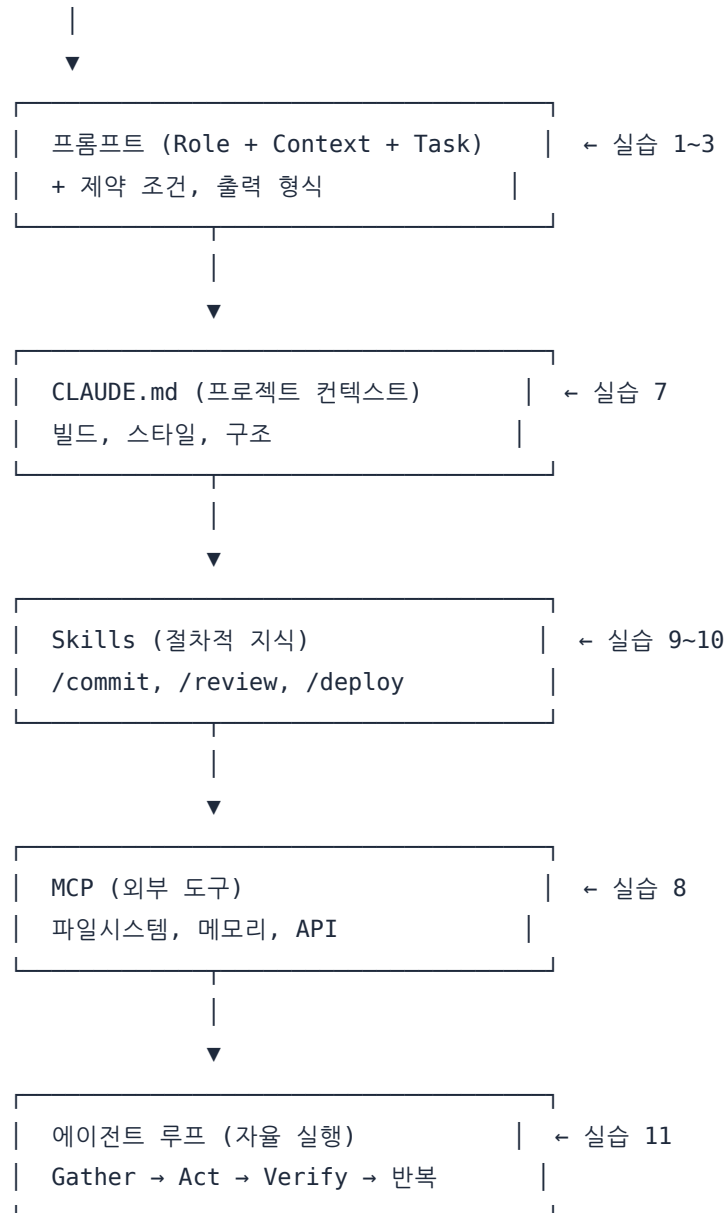
차시	주제	배운 것
1 3	프롬프트 기초	Role + Context + Task, Few-shot, CoT
4	프롬프트 한계	환각, 프롬프트만의 한계를 체험
5 6	컨텍스트 설계	시스템 프롬프트, 대화 관리
7	CLAUDE.md	프로젝트 설정 파일로 AI 행동 제어
8	MCP	외부 도구 연결 (파일시스템, 메모리)
9 10	Skills	나만의 전문가 만들기, 복합 스킬
11	에이전트 루프	대규모 작업, Gather/Act/Verify

차시	주제	배운 것
12	워크플로우	모든 것을 조합한 실무 자동화

이 흐름은 우연이 아니에요. 의도적으로 “기초 → 한계 인식 → 도구 → 조합”의 순서로 구성했어요.

기술 스택 전체도

여러분의 요청



각 계층이 아래 계층 위에 쌓이는 구조예요. 프롬프트 없이 MCP만 있으면 의미가 없고, CLAUDE.md 없이 에이전트 루프를 돌리면 일관성이 떨어져요. **전체가 조화를 이룰 때 진정한 가치가 나와요.**

AI 자동화의 핵심은 “하나의 완벽한 도구”가 아니라 “여러 도구의 조합”이에요. 프롬프트가 기초를 잡고, CLAUDE.md가 맥락을 주고, MCP가 능력을 확장하고, Skills가 절차를 정의하고, 에이전트 루프가 자율적으로 실행해요.

실습편 전체 핵심 정리

- **프롬프트**: AI와 소통하는 가장 기본적인 방법이에요. 구조화하면 품질이 올라가요
- **CLAUDE.md**: 프로젝트 단위로 AI의 행동을 일관되게 제어해요
- **MCP**: AI에게 외부 도구를 연결하는 표준 프로토콜이에요
- **Skills**: 반복되는 작업 절차를 파일로 정의해서 한 번의 명령으로 실행해요
- **에이전트 루프**: 분석 → 실행 → 검증을 자동으로 반복해요
- **워크플로우**: 이 모든 것을 조합해서 실무 업무를 자동화해요
- **검증은 필수**: 자동화 결과를 검증하는 방법이 없으면 자동화하지 마세요

트러블슈팅

Q. 자동화할 업무가 생각나지 않아요

A. 이번 주에 가장 귀찮았던 작업을 떠올려보세요. '또 이걸 해야 해?'라고 생각한 작업이 자동화의 1순위 후보예요. 메일 정리, 보고서 초안, 코드 리뷰 같은 반복 작업부터 시작하세요.

Q. 워크플로우가 매번 다른 결과를 내놓아요

A. Skill의 절차를 더 구체적으로 작성하세요. '적절히 분석하세요' 대신 '아래 5가지 항목을 순서대로 점검하세요'처럼요. 또한 CLAUDE.md에 출력 형식을 엄격하게 지정하세요.

Q. 워크플로우가 너무 복잡해져서 관리가 어려워요

A. 하나의 워크플로우가 5단계를 넘기면 2~3개의 스킬로 분리하세요. 각 스킬은 하나의 책임만 가지고, 상위 스킬이 이들을 조합하는 구조가 유지보수에 유리해요.

Q. AI가 만든 결과물의 품질이 기대에 못 미쳐요

A. 첫 버전은 80점이면 성공이에요. 피드백을 반영해서 Skill과 CLAUDE.md를 점진적으로 개선하세요. 3~5번의 반복 개선 후에 90점 이상을 기대할 수 있어요.

Q. 동료에게 워크플로우를 공유하고 싶어요

A. .claude 폴더와 CLAUDE.md를 Git에 커밋하면 팀원도 동일한 환경을 사용할 수 있어요. WORKFLOW.md 문서까지 포함하면 온보딩이 훨씬 수월해요.

실습 체크리스트

- ☐ 반복 업무 1개를 선택하고 분석 워크시트를 작성했다
- ☐ 프롬프트 + 컨텍스트 + 도구 + 절차 + 검증의 5요소를 설계했다
- ☐ 설계한 워크플로우를 실제로 실행하고 결과를 확인했다
- ☐ WORKFLOW.md 문서를 작성했다

- 여기까지 완료하면 이번 실습은 성공이에요! -

- ☐ [도전] 워크플로우를 3번 반복 실행하고 재현성을 확인했다
- ☐ [도전] 수동 대비 시간 절감 비율을 측정했다
- ☐ [도전] 알려진 한계와 개선 계획을 문서에 포함했다

실습편을 마치며

12개의 실습을 통해 AI를 “대화 상대”에서 “업무 파트너”로 바꾸는 여정을 함께했어요.

프롬프트 하나로 시작해서, CLAUDE.md로 맥락을 주고, MCP로 도구를 연결하고,
Skills로 절차를 정의하고, 에이전트 루프로 자율 실행하는 전체 그림을 완성했어요.

기술은 계속 변해요. 새로운 모델, 새로운 도구, 새로운 프로토콜이 등장할 거예요.
하지만 이번 실습에서 익힌 “설계하고, 실행하고, 검증하고, 개선하는” 사이클은 변하지 않아요.

여러분의 AI 워크플로우 여정은 여기서 끝이 아니라 시작이에요.